RISQrypt - Fast, Secure and Agile Hardware-Software Co-Design for Post-Quantum Cryptography

Tolun Tosun[1], Atıl Utku Ay[1], Quinten Norga[2], Suparna Kundu[2], Melik Yazıcı[1], Erkay Savaş[1] and Ingrid Verbauwhede[2]

[1] Sabanci University, Istanbul, Türkiye
[2] COSIC, KU Leuven, Leuven, Belgium

Abstract. In this paper, we present RISQrypt, the first unified architecture in the literature that implements Kyber (ML-KEM) and Dilithium (ML-DSA), standardized lattice-based Post-Quantum Cryptography (PQC) algorithms, with masking. RISQrypt is a hardware–software co-design framework that integrates dedicated cryptographic accelerators to speed up polynomial arithmetic, hashing, and mask-conversion operations, the latter being one of the primary bottlenecks in masked implementations of lattice-based PQC. Our design achieves low latency while providing both theoretical and practical side-channel security, as validated through experimental evaluation. Specifically, the masked decapsulation of Kyber768 requires **109K** clock cycles, while masked signing of Dilithium3 requires **1230K** clock cycles on average. These results demonstrate **11.3**× time-performance improvement over existing masked implementations. Our time performance results for unprotected functions also outperform the existing work; achieving speed-ups of **10.64**× and **8.94**× for Kyber's encapsulation and decapsulation algorithms, respectively, and **1.14**× and **2.23**× or Dilithium's signature generation and verification algorithms, respectively, when compared to existing designs with similar resource usage. In addition, prior designs are more limited in scope, generally supporting only a single scheme and lacking either the side-channel protection or the unified, crypto-agile framework that enables support for both Kyber and Dilithium as in our architecture. Leveraging the HW/SW co-design approach, our proposed architecture can be readily extended to support other PQC standards such as Falcon and SPHINCS+, as well as algorithms sharing similar computational building blocks, through firmware reprogramming.

Keywords: Masking · Kyber · Dilithium · Side-Channel Analysis · Hardware Acceleration · Lattice-Based Cryptography · RISC-V

1 Introduction

The security of currently deployed public-key cryptosystems depends on the computational hardness of number theory-based problems, such as the (elliptic curve) discrete logarithm problem used in the Diffie-Hellman key exchange [DH76, Kob87] and digital signatures [Sch90], as well as the integer factorization problem underlying RSA [RSA78]. Although they are conjectured to be resistant to all currently known cryptanalytic methods on classical computers, research has demonstrated that Shor's algorithm [Sho94] could efficiently solve them in polynomial time if a sufficiently powerful quantum computer were available.

To address the emerging security challenges posed by quantum computing, the National Institute of Standards and Technology (NIST) initiated a standardization process for post-quantum public-key cryptographic (PQC) algorithms in 2016. This initiative aims to establish quantum-resistant cryptographic standards, including digital signature algorithms (DSAs) and key encapsulation mechanisms (KEMs).

As of now, NIST has selected two KEMs: CRYSTALS-Kyber (lattice-based) and HQC (code-based), alongside three DSAs: CRYSTALS-Dilithium (lattice-based), Falcon (lattice-based), and SPHINCS+ (hash-based). Lattice-based cryptographic (LBC) constructions [Reg05, Ajt96], relying on the computational hardness of lattice problems, have emerged as leading candidates for post-quantum security, offering an exceptional balance between efficiency, scalability, and provable robustness against quantum adversaries.

Within this category, CRYSTALS-Kyber relies on the Module Learning With Errors (MLWE) problem, while CRYSTALS-Dilithium is based on both MLWE and the Module Short Integer Solution (MSIS) problems. As members of the CRYSTALS family, Kyber and Dilithium share core structural similarities and employ closely related computational primitives, which enable their seamless integration within a unified implementation framework.

As noted in both the CRYSTALS-Kyber and CRYSTALS-Dilithium submissions, the main sources of performance bottlenecks arise from hash computations and multiplication operations in polynomial rings. A widely adopted approach to alleviating these bottlenecks is hardware acceleration, in which dedicated and optimized hardware units offload computationally intensive tasks from the host general-purpose processor.

Keccak [BDPA12] is a cryptographic algorithm that serves as the foundation of the SHA-3 (Secure Hash Algorithm 3) standard, introduced by NIST. Its flexible sponge construction allows Keccak to support both fixed-length hash functions and extendable-output functions (XOFs), which can produce outputs of arbitrary length. This versatility makes it particularly well-suited for a broad range of cryptographic applications, including lattice-based schemes. Hardware acceleration of Keccak is a mature research area, with numerous well-established designs reported in the literature such as [GSM17, ZSS+21].

To enhance the efficiency of polynomial multiplication in lattice-based cryptographic algorithms, the Number Theoretic Transform (NTT) [AB75], an integer variant of the Discrete Fourier Transform (DFT), is commonly employed to improve computational performance. In the literature, a multitude of studies have proposed hardware accelerators for the NTT operation [FVR+22, MBB+23, NDMZ+21, MCL+23, WZZ+24, WWL+24, XL21, HHLW20, GLK21, DMMM23, JGCS24, KNAH22, ZZW+22, BNG21, WZCG22, CKRG+24]. While several of these works integrate support directly into the processor data-path using a tightly coupled approach [MBB+23, NDMZ+21, WWL+24], others adopt a loosely coupled design in which the accelerator operates as an independent unit connected to the processor core [FVR+22, WZZ+24, MCL+23]. Although the NTT constitutes the primary bottleneck in polynomial arithmetic, accelerating the remaining operations, including polynomial additions and subtractions, point-wise multiplications, divisions, pseudo-random sampling, and encoding/decoding, is also essential to achieve state-of-the-art performance.

Side-channel attacks (SCAs) pose significant security threats to the implementation of cryptographic algorithms, with masking serving as the primary countermeasure. As PQC algorithms have only recently been standardized, their resistance to side-channel attacks has been investigated in a relatively limited number of studies. For example, in [FVR+22], dedicated operations required for masked implementations were integrated into the instruction set architecture (ISA) to enhance resilience against SCAs.

There are two main masking types: arithmetic masking and Boolean masking. Many cryptographic schemes, especially the lattice-based PQC, require the use of both methods simultaneously, necessitating frequent and secure conversions between them (A2B & B2A). These conversions, however, are computationally intensive and resource consuming, underscoring the importance of accelerating masking conversions to maintain both security and efficiency. Consequently, A2B & B2A conversions emerge as a third major bottleneck

Table 1: Overview of efficient implementations of lattice-based PQC in the literature.

Work	Supported Scheme	Class	SCA Protection	Hardware Acceleration	Agility
RISQrypt (This Work)	K, D	HW/SW	✓	✓	✓
[FVR+22]	K	HW/SW	✓	✓	✓
[WZZ+24]	K, D	HW/SW	✢	✓	
[KSFS24]	D	HW/SW		✓	✓
[MBB+23, NDMZ+21]	K, D	HW/SW		✓	
[DMMM23, WWL+24]	K	HW/SW		✓	
[MCL+23]	D	HW/SW		✓	
[AOP+25]	K, D	HW/SW		✓	✓
[JGCS24, KNAH22]	K	HW	✓	✓	
[BAA23]	K, D	HW	❖	✓	
[XL21, HHLW20, GLK21]	K	HW		✓	
[ZZW+22, BNG21, WZCG22]	D	HW		✓	
[CKRG+24, AMI+23]	K, D	HW		✓	
[BC22, HKL+22, CGMZ23]	K	SW	✓		✓
[CGL+24, CGTZ23]	D	SW	✓		✓

✢ SCA protection limited to polynomial arithmetic. ❖ SCA protection limited to Kyber.
K: Kyber, D: Dilithium

in masked implementations of lattice-based PQC, alongside polynomial arithmetic and hashing.

RISC-V is an open and extensible ISA developed at UC Berkeley. Its modular RISC design supports a wide range of platforms, from simple embedded systems to high-performance and energy-efficient computing. Owing to its open-source and extensible nature, many cryptographic acceleration works build on RISC-V cores [FVR+22, MBB+23, NDMZ+21, WZZ+24, MCL+23]. In the *loosely coupled* approach in hardware acceleration, the hardware unit operates as an independent module connected to the processor via a data bus. In this configuration, the unit typically includes its own cache memory or register file, as well as a dedicated control unit. This approach preserves the original ISA, ensuring software compatibility and simplifies integration into existing toolchains and hardware platforms. On the other hand, in the *tightly coupled* approach in hardware acceleration, a specialized hardware unit is integrated directly into the processor's datapath and is accessed through custom instruction(s). While this approach achieves low communication latency and minimal control overhead, it requires modifications to the original ISA and typically provides more limited acceleration gains compared to loosely coupled accelerators, unless vector instructions with significant area overhead are used as in [AOP+25]. Our proposed solution, **RISQrypt**, is based on a HW/SW co-design framework and relies on dedicated, loosely coupled hardware accelerators for hashing, polynomial arithmetic, and mask conversions.

Related Work. Several studies have investigated the efficient implementation of lattice-based PQC schemes, some of which are summarized in Table 1. These works can be broadly categorized into three groups: (i) software-only (SW), (ii) hardware-only (HW), and (iii) hardware–software co-design (HW/SW) solutions. SW solutions are agile, as the software can be re-compiled and re-programmed, but they inherently lack hardware acceleration and therefore cannot achieve high performance. Here, we use the term agility to refer ability of a design to support different parameter sets (e.g., moduli or polynomial degrees) either at run time or at software compile time, without requiring changes on the hardware at design time, where modifications are costly due to tape-out. HW solutions, on the other hand, offer high performance but are typically not agile, since algorithmic steps and parameters are hard-coded for Kyber or Dilithium. HW/SW co-design approaches enable the combination of agility and high performance; however, some existing works still fix their implementations to Kyber, Dilithium, or both. Overall, existing solutions either lack a unified architecture supporting both Kyber and Dilithium, do not address side-channel security, or provide only limited protection.

Contributions. In this work, we address all these challenges and close the existing gap in the literature. In summary:

- For the first time in the literature, we present a unified architecture that implements both Kyber and Dilithium with integrated side-channel protection, fully covering all sensitive functions. The proposed design is unique in that it is agile: it can be reprogrammed to support Kyber or Dilithium with different side-channel countermeasures, or even to implement other algorithms built from similar computational blocks. We provide an in-depth security and performance analysis of the architecture. The results show that our solution achieves superior performance, flexibility, and security. Although no prior work provides the same set of qualitative features, our design still outperforms existing implementations that support significantly fewer functionalities by up to an order of magnitude in both time performance and area–time product.

- We propose an efficient and fully programmable polynomial arithmetic accelerator, NTT-Lite, designed to support the core operations of Kyber and Dilithium, as well as a wide range of cryptographic schemes based on modular and polynomial arithmetic. We show that the accelerator's configurable architecture not only enables extensibility beyond Kyber and Dilithium, but is also essential for implementing side-channel countermeasures, particularly so-called gadgets. The accelerator features a dual-mode operation that efficiently supports both full- and half-word modes, as required by Dilithium and Kyber, respectively.

- We propose a general-purpose, efficient, and high-throughput mask conversion accelerator (A2B & B2A), which can be used to implement protected versions of Kyber and Dilithium as well as other algorithms. We show that the required gadgets for masked Kyber and Dilithium can be implemented exclusively using the proposed mask conversion accelerator, together with NTT-Lite, without the need for additional operations at the software level. Our study presents the design decisions associated with the masked implementations of the non-linear operations in both schemes, given a high-throughput mask conversion accelerator. We also show that these design and implementation choices yield efficient masked implementations for Kyber and Dilithium. We provide an in-depth side-channel security analysis for secret key-involved functions in Kyber and Dilithium. The security of our masked gadgets is verified theoretically and practically through the Test Vector Leakage Assessment (TVLA) methodology.

2 Preliminaries

This section briefly introduces the notation and reviews the essential background on the CRYSTALS PQC family, namely Kyber and Dilithium schemes, the number-theoretic transform, and the fundamentals of boolean masking for side-channel protection. In the interest of space, the full algorithmic details of Kyber and Dilithium are provided in

Appendix A. In this section, we briefly highlight the supporting functions of both schemes, which play a key role in our design.

2.1 Notation

Throughout the paper, bold letters, such as **a** are used to denote vectors and matrices. The elements of vectors and matrices are referred using square brackets (e.g., $\mathbf{a}[i]$ or $\mathbf{a}[i][j]$). $\mathcal{B}^l$ denotes the set of l-byte strings. The i-th bit of integers are accessed using square brackets, such as $a[i]$ for $a \in \mathbb{Z}_{2^k}$ and $0 \leq i < k$. The $||$ operator denotes concatenation. The symbol $\leftarrow$ denotes uniform random sampling from the set on the right-hand side, e.g., $a \leftarrow \mathbb{Z}$.

The polynomial ring $\mathcal{R}_q = \mathbb{Z}_q/(x^n + 1)$ consists of polynomials with coefficients in $\mathbb{Z}_q$, which form residue class obtained by the division by the cyclotomic polynomial $x^n + 1$, where n is a power of two.

The notations $s^{\{0:d\}_A}$ and $s^{\{0:d\}_B}$ denote the complete set of shares of a d^{th}-order arithmetic- and boolean-masked variable s, respectively. When a single index appears in the superscript, such as $s^{\{i\}_A}$, it refers specifically to share-i. If $s^{\{0:d\}_A}$ (or $s^{\{0:d\}_B}$) is provided as input or output of a function, the function receives or produces all shares of the variable. Likewise, when this notation appears in an arithmetic or boolean operation, it indicates that the same operation is applied independently to each share.

For A2B & B2A conversions, we use a subscript to indicate the modulus type. For example, B2A_{2^k} denotes a Boolean-to-arithmetic conversion with a power-of-two modulus, whereas B2A_q denotes the conversion modulo an arbitrary modulus, not necessarily a power of two.

2.2 CRYSTALS-Kyber

CRYSTALS-Kyber is a lattice-based KEM built upon the hardness of the MLWE problem, which offers a favorable balance between security and efficiency. The scheme operates over the polynomial ring $\mathcal{R}_q$ with $n = 256$ and modulus $q = 3329$. By exploiting structured module lattices, Kyber achieves compact key sizes and efficient polynomial arithmetic. Polynomial multiplications are accelerated using the NTT, and noise terms are sampled from centered binomial distributions to ensure both performance and security.

Kyber was originally introduced as a Chosen Plaintext Attack (CPA)-secure public-key encryption scheme, known as Kyber.CPAPKE. This base encryption scheme provides semantic security under the MLWE assumption, and it serves as the foundation for the KEM construction. The algorithms used in Kyber.CPAPKE are shown in Figure 11.

The functions used in Kyber.CPAPKE can be briefly explained as follows. Parse and Decode functions are used to map a sequence of bits (message, key, output of a hash function) to polynomial coefficients in $\mathbb{Z}_q[x]$. Encode function is the inverse of Decode function. G is a hash-based generator that expands a seed into multiple cryptographic values, PRF is a keyed pseudorandom function used to deterministically generate secret noise polynomials, and XOF is an extendable-output function used to sample uniformly random polynomial coefficients from a seed. All these functions are specified in [BDK+22] and implemented using SHA-3 hash function. CBD_η denotes centered binomial distribution in range $[-\eta, \eta]$, where η is a small integer. $\mathsf{Compress}_q(x, d)$ and $\mathsf{Decompress}_q(x, d)$ functions are used to reduce the size of the ciphertext and perform LWE error correction in Kyber. $\mathsf{Compress}_q(x, d)$ function (defined in Eq. 1) takes an element $x \in \mathbb{Z}_q$ and output an integer in the interval $[0, 2^d)$.

$$\mathsf{Compress}_q(x, d) = \left\lceil \frac{2^d}{q} \cdot x \right\rfloor \bmod 2^d, \tag{1}$$

where $\lceil x \rfloor$ denotes rounding x to the nearest integer. $\mathsf{Decompress}_q(x, d)$ function is simply the opposite of $\mathsf{Compress}_q(x, d)$ function defined as follows

$$\mathsf{Decompress}_q(x, d) = \left\lceil \frac{q}{2^d} \cdot x \right\rfloor. \tag{2}$$

The CCA-secure variant of CRYSTALS-Kyber, called Kyber.CCAKEM, is obtained by applying the Fujisaki–Okamoto (FO) transformation to the Kyber.CPAPKE. This conversion provides resistance against adaptive chosen-ciphertext attacks while preserving the efficiency of the underlying MLWE-based construction. In this design, a shared secret is derived by hashing both the ciphertext and an ephemeral secret, and a verification step ensures robustness against ciphertext tampering. The algorithms used in Kyber.CCAKEM is shown in Figure 12, where H, G, and KDF are specified in [BDK+22] and implemented using SHA-3.

2.3 CRYSTALS-Dilithium

CRYSTALS-Dilithium is a lattice-based digital signature scheme relying on the hardness of the MLWE and MSIS problems. It achieves strong security through structured lattices and rejection sampling, avoiding the need for complex Gaussian sampling. With multiple parameter sets targeting different security levels, Dilithium provides efficient key generation and signing while maintaining resistance to classical and quantum attacks. The algorithms used in Dilithium is shown in Figure 13. Dilithium employs parameters $n = 256$ and $q = 8380417$ over the ring $\mathcal{R}_q$.

The functions used in Dilithium can be briefly explained as follows. The functions H, CRH, ExpandA, and ExpandMask are specified in [DKL+22] and implemented using SHA-3. S_η denotes uniform sampling in $[-\eta, \eta]$, where η is a small integer. $\mathsf{Power2Round}_q(t, d)$ function returns t_1 and t_0, where $t \in \mathbb{Z}_q$, $t = t_1 \cdot 2^d + t_0$, and $t_0 \in (-q/2, q/2]$.

The function $\mathsf{Decompose}_q(r, \alpha)$ can be defined as follows. Let r be an integer in $\mathbb{Z}_q$, then we can represent r as

$$r = r_1\alpha + r_0, \tag{3}$$

where $q - 1 = m \cdot \alpha$, $r_1 \in [0, m]$, and $r_0 \in (-\alpha/2, \alpha/2]$. The function returns r_1, and r_0. $\mathsf{HighBits}_q(r, \alpha)$ and $\mathsf{LowBits}_q(r, \alpha)$ functions invoke $\mathsf{Decompose}_q(r, \alpha)$, and return r_1 and r_0, respectively. $\mathsf{MakeHint}_q(z, r, \alpha)$ checks whether z generates a carry bit for $\mathsf{HighBits}_q(r, \alpha)$ or not. $\mathsf{UseHint}_q(h, r, \alpha)$ adds the carry to $\mathsf{HighBits}_q(r, \alpha)$ depending on h, which denotes the hint vector produced by $\mathsf{MakeHint}_q(z, r, \alpha)$.

2.4 Number Theoretic Transform (NTT)

NTT is a modular analogue of the DFT, commonly employed in PQC and FHE to accelerate polynomial multiplications in $\mathcal{R}_q$. Unlike the DFT, which operates over the complex numbers, NTT is performed over a finite field or ring, typically $\mathbb{Z}_q$, where q is a prime admitting a principal $2n$-th root of unity, ψ. For a polynomial $a(x)$ of degree $n - 1$, the NTT maps its coefficient vector to the evaluation domain by computing

$$\bar{a}_k = \sum_{j=0}^{n-1} a_j \psi^{j(2k+1)} \bmod q, \quad 0 \le k < n, \tag{4}$$

where ψ is a primitive $2n$-th root of unity modulo q.

In the time domain, multiplying two degree-$(n - 1)$ polynomials directly requires $\mathcal{O}(n^2)$ coefficient multiplications and additions. By applying NTT, the polynomials are transformed into the frequency domain, where multiplication reduces to element-wise

Algorithm 1 Barrett Reduction Variant 1

Input: word-size λ
Input: $a \in [0, 2^{2\lambda})$
Input: $q \in (0, 2^{\lambda})$, $\mu = \lfloor 2^{2\lambda}/q \rfloor$
Output: $r = a \mod q$
Output: $a = t_1 \cdot q + r$
1: $t_0 = (a \cdot \mu) \gg 2\lambda$
2: $t_1 = t_0 \cdot q \mod 2^{\lambda+1}$
3: $r = a - t_1 \mod 2^{\lambda+1}$
4: **if** $r \geq q$ **then**
5: $\quad r = r - q$, $t_0 = t_0 + 1$
6: **end if**
7: **return** r, t_0

Algorithm 2 Barrett Reduction Variant 2

Input: word-size λ
Input: $a \in [0, 2^{2\lambda})$
Input: $q \in [2^{\lambda-1}, 2^{\lambda})$, $\mu = \lfloor 2^{2\lambda}/q \rfloor$
Output: $r = a \mod q$
Output: $a = t_1 \cdot q + r$
1: $a_1 = a \gg \lambda$
2: $t_0 = (a_1 \cdot \mu) \gg \lambda$
3: $t_1 = t_0 \cdot q \mod 2^{\lambda+2}$
4: $r = a - t_1 \mod 2^{\lambda+2}$
5: **if** $r \geq q$ **then**
6: $\quad r = r - q$, $t_0 = t_0 + 1$
7: **end if**
8: **if** $r \geq q$ **then**
9: $\quad r = r - q$, $t_0 = t_0 + 1$
10: **end if**
11: **return** r, t_0

multiplication, which is only $\mathcal{O}(n)$. The Fast Fourier Transform (FFT) algorithm, adapted to modular arithmetic, enables the efficient computation of the NTT and its inverse (INTT) in $\log n$ stages, each consisting of $n/2$ butterfly operations, resulting in an overall complexity of $\mathcal{O}(n \log n)$. Due to the convolution theorem, multiplication in the time domain (i.e., multiplication in $\mathcal{R}_q$) can be performed using element-wise multiplication in the NTT domain.

The Incomplete NTT is a variant of the standard NTT, which may be used to reduce computational overhead and memory usage in lattice-based cryptosystems. Unlike the full NTT, which performs all $\log_2 n$ butterfly stages in both forward and inverse directions, the incomplete version omits one or more of the stages when the algorithmic structure allows. Subsequent element-wise multiplication, then, becomes more involved and should be adapted to operate on this partially transformed data. Incomplete NTT is sometimes necessary due to the mathematical structure of $\mathcal{R}_q$, as in Kyber, where a $2n$-th root of unity does not exist. It may also be preferred for efficiency reasons in Dilithium [AHKS22].

2.5 Barrett Reduction

Barrett reduction [Bar87b] is an efficient algorithm for computing the remainder $r = a \bmod q$, where $a = q \cdot t_0 + r$ for two integers a and q. The algorithm first computes a quotient t_0, and then derives the remainder as $a - t_0 \cdot q$. We use two variants of Barrett reduction in Algorithm 1 and Algorithm 2. The first variant incurs higher cost, as it requires a $2\lambda \times 2\lambda$-bit multiplication (Line 1), whereas the second variant only involves a $\lambda \times \lambda$-bit multiplication (Line 2). However, the former supports reduction of inputs of arbitrary size given the word size λ. In contrast, the second variant assumes that the input a is at most twice the bit-length of the modulus q (i.e., the divisor).

2.6 Arithmetic and Boolean Masking

Masking is a widely applied countermeasure designed to secure cryptographic hardware and software implementations against passive side-channel attacks, such as those exploiting electromagnetic (EM) emissions or power consumption. Based on the secret sharing principle introduced by Shamir [Sha79], a sensitive variable s is decomposed into $d+1$ uniformly random shares $s^{\{i\}}$, where $0 \leq i \leq d$, ensuring d-th order probing security [ISW03].

In order to prove the probing security of larger gadgets, which are small, composable circuits or algorithmic components that operate on masked shares, Barthe et al. [BBD+16] introduce security notions of (strong-) non-interference (t-(S-)NI) for proving the security of smaller gadgets and their composition.

Definition 1 (t-(Strong) Non-Interference [BBD+16,SPOG19])**.** A gadget with one output sharing and $m^{\{i\}}$ input shares is t-NI (resp. t-SNI) secure if any set of at most t_1 probes on its internal wires and t_2 probes on output sharings such that $t_1 + t_2 \leq t$ can be simulated with $t_1 + t_2$ (resp. t_1) shares of each of its $m^{\{i\}}$ input shares.

Both of the following masking techniques are employed in lattice-based PQC implementations to ensure side-channel resistance: arithmetic masking and boolean masking. For arithmetic masking, the sensitive value $s \in \mathbb{Z}_q$ is divided into uniformly random $d+1$ shares $s^{\{i\}} \in \mathbb{Z}_q$, and s is the arithmetic sum of $d+1$ shares, i.e., $s = \sum_{i=0}^{d} s^{\{i\}_A} \mod q$. For boolean masking, s is the result of the bitwise XOR of $d+1$ shares, i.e., $s = \oplus_{i=0}^{d} s^{\{i\}_B}$. In masking, each individual share is statistically independent of the original secret. Consequently, an attacker must have/gain knowledge of all $d+1$ shares to infer any meaningful information about the underlying secret s. As both arithmetic and boolean masking are used in lattice-based schemes such as digital signature algorithms and key encapsulation mechanisms, it becomes necessary to transform shares between these masking domains, which incurs significant performance overhead. Arithmetic-to-Boolean (A2B) denotes the transformation of arithmetic shares into boolean shares, while Boolean-to-Arithmetic (B2A) refers to the inverse operation.

Refresh is a frequently used gadget to satisfy specific security requirements (e.g., achieving t-SNI) by re-randomizing the shares of sensitive variables. Intuitively, this operation prevents the propagation of dependencies on previously used randomness across different gadgets. Formally, Refresh combines its input with a fresh sharing of zero in the corresponding domain. That is, $s'^{\{0:d\}_B} \leftarrow s^{\{0:d\}_B} \oplus z^{\{0:d\}_B}$ in the Boolean domain (or $s'^{\{0:d\}_A} \leftarrow s^{\{0:d\}_A} + z^{\{0:d\}_A}$ in the arithmetic domain), where $z^{\{0:d\}_B}$ (or $z^{\{0:d\}_A}$) is freshly generated sharing of zero ($z = 0$).

3 Hardware Accelerators

In this section, we provide an overview of the cryptographic hardware accelerators used in RISQrypt. The general architecture is shown in Figure 1. The design comprises three cryptographic accelerators, a polynomial arithmetic accelerator referred to as **NTT-Lite**, a **Keccak** accelerator, and an A2B/B2A mask conversion accelerator referred to as **X2X**. Cryptographic accelerators access the data RAM via a shared Direct Memory Access (DMA) interface to improve data read and write efficiency. Memory-Mapped I/O (MMIO) is a standard master–slave communication abstraction used to access peripherals. Although NTT-Lite also operates on masked data, it performs only linear operations and can therefore process each share independently using a single DMA interface. In contrast, Keccak and X2X include non-linear operations that require simultaneous access to multiple shares, necessitating separate DMA interfaces per share to prevent side-channel leakage that could result from multiplexing multiple shares on a single interface. For Keccak and X2X, multiple DMA interfaces are never active at the same time. In our implementation, although both MMIO interface and the DMA interface are realized using the Wishbone protocol, the design is not tied to this choice. For example, these interfaces could alternatively be implemented using AXI, also a widely adopted standard. Similarly, although RISC-V is used as the main processor in our implementation, this choice is also not mandatory, and our work is orthogonal to the specific software core. In this work, we employ the 32-bit HORNET RISC-V Core[1] presented in [YTO21].

[1] https://github.com/yavuz650/RISC-V **commit id:** 6c7af88

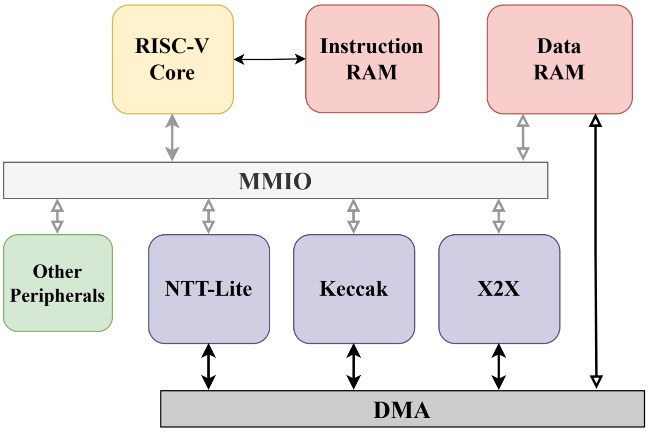
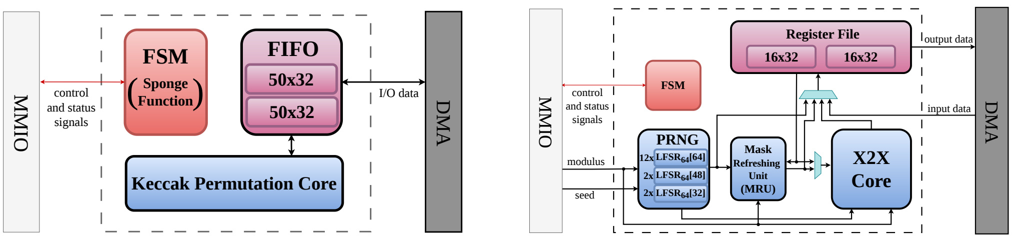


Figure 1: System Architecture



(a) Keccak (b) X2X

Figure 2: Keccak and X2X accelerators high-level architecture. Black lines denote data paths, red lines denote control and status signals. Internal control signals are omitted for clarity.

We highlight that our work combines dedicated cryptographic accelerators (highlighted in light blue in Figure 1 with a software layer that leverages these accelerators. Consequently, the proposed approach is independent of the overall system architecture, provided that the accelerators can be efficiently accessed by the processor and that the processor can be programmed to execute our PQC software. Finally, while our approach supports arbitrary-order masking, we describe the architecture here for the first-order case to keep the presentation simple.

As discussed in Section 2, polynomial arithmetic is the fundamental operation in LBC algorithms, and its efficient implementation is critical to enhance the performance. We present the proposed polynomial arithmetic accelerator NTT-Lite in Section 4. In the next subsections, we detail Keccak and X2X accelerators.

3.1 Keccak Accelerator

Figure 2a presents the simplified architecture of the Keccak Accelerator. We integrated the masked Keccak permutation core proposed in [GSM17], based on the so-called domain-oriented masking approach, into our architecture. We should note that the integrated Keccak core only provides the Keccak permutation. Implementing higher-level hash functions such as SHA-3 or SHAKE requires to implement the so-called sponge construction [D+15]. In our design, the sponge construction is implemented directly within the accelerator to minimize software interaction and thereby improve overall performance.

Furthermore, a FIFO mechanism is incorporated to overlap the Keccak permutation and data input and output. The level of parallelism in the Keccak permutation core is configurable, and we configured it such that it finishes a Keccak permutation in $4 \times 24 = 96$ clock cycles. Masking can be enabled or disabled at run time via control signals, allowing improved efficiency for non-sensitive hashing operations.

3.2 X2X Accelerator

Applying masking countermeasures to lattice-based cryptographic schemes necessitates the use of both arithmetic and Boolean masking [MGTF19, ÖY23, KDVB+22, CGTZ23]. In particular, core operations such as polynomial addition and multiplication are performed over arithmetic shares, while bitwise operations—such as those used in the Keccak hash function—are more naturally protected using Boolean masking. Consequently, a secure implementation must provide an efficient and reliable mechanism to convert between these two masking domains, specifically through Arithmetic-to-Boolean (A2B) and Boolean-to-Arithmetic (B2A) transformations.

To accelerate A2B and B2A conversions, we integrate a dedicated unit into our architecture. Specifically, we adopt the X2X core from [NDKV25], which also lends its name to our cryptographic accelerator. The X2X core enables mask conversion with high-throughput, such as 1 to 2 conversions per cycle, supporting both A2B and B2A conversions, for both a power-of-two or prime moduli. We would like to note that X2X core was originally designed for a word size of 13-bit in [NDKV25], and it is expanded to 32-bit for this work. The X2X core employs a 13-stage pipeline to support high-throughput A2B and B2A conversions. When the modulus is a power of two, the core supports a dual mode in which two conversions can be performed simultaneously, thereby doubling the throughput.

Table 2: Operations supported by X2X accelerator.

	Active Units			
Opcode	**PRNG**	**MRU**	**X2X Core**	**Modes**
OP_PRNG	✓	✗	✗	
OP_X2X	✓	✗	✓	A2B, B2A, $\text{B2A}_{\text{1-bit}}$
OP_REF	✓	✓	✗	A, B
OP_MASK	✓	✓	✗	A, B
OP_REFX2X	✓	✓	✓	A2B, B2A, $\text{B2A}_{\text{1-bit}}$

A: Arithmetic, B: Boolean

The simplified architecture of X2X accelerator is given in Figure 2b. As shown, in addition to the X2X core, we integrated a pseudo-random number generator (PRNG), and a mask refresher unit (MRU).

4 NTT-Lite

In this section, we provide a detailed description of the architecture and sub-modules of our polynomial arithmetic accelerator NTT-Lite.

4.1 Scope

NTT-Lite is designed to accelerate not only the NTT operation but also other polynomial arithmetic and supporting functions used in PQC schemes, such as compression, encoding and pseudo-random sampling, which cannot be efficiently implemented using software.

Although this work focuses on secure implementations of Kyber and Dilithium, NTT-Lite is not restricted to the parameters of these schemes. It supports modulus q sizes up to 32-bit and polynomial degrees n of up to 256, ensuring an agile design, flexibility and reusability across different LBC algorithms.

On the other hand, masked implementations of Kyber and Dilithium require configurations that deviate from the standard parameter sets (e.g., $q = 3329$ or $q = 8380417$), such as the moduli of these schemes. This flexibility is particularly necessary for implementing the arithmetic involved in their non-linear operations. Our configurable NTT design satisfies these requirements. Additional discussion on the role of NTT-Lite in masked implementations is provided in Section 6.2. We employ unsigned arithmetic over $\mathbb{Z}_q$ as signed arithmetic makes differential SCA attacks significantly easier [TMS24].

4.2 Architecture

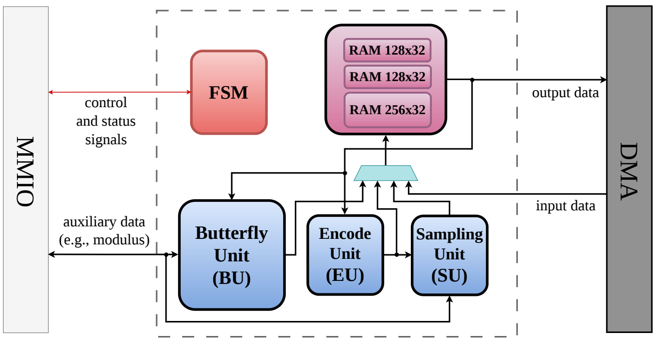


Figure 3: NTT-Lite high-level architecture. Internal control signals are omitted for clarity.

Figure 3 illustrates the high-level and simplified architecture for NTT-Lite. As shown, the NTT-Lite contains of two 128×32 and one 256×32 RAM blocks. The core arithmetic unit in NTT-Lite is the Buttefly Unit (BU) as in almost all NTT designs in the literature. In addition, an Encode Unit (EU) is integrated into the design to accelerate the encoding and decoding operations used in PQC algorithms. Moreover, a Sampling Unit (SU) is implemented to perform operations related to pseudo-random number sampling used in PQC schemes. FSM performs address calculations and generates control signals for the internal memories as well as BU, EU and SU.

Since a butterfly circuit in the NTT computation has two inputs and two outputs, we use two independent 128×32 RAMs. In this manner, NTT-Lite can feed BU with a valid pair of inputs at each clock cycle. The 256×32 RAM is used for the storage of twiddle factors in NTT, as well as one of the operands in certain operations. We detail the internal RAM usage in the following sections. In addition to the main data path, an auxiliary 160-bit data path is employed. This path delivers constant values required by the provided functionalities, such as the modulus used in modular arithmetic. Further details on this auxiliary constant-data are provided in the next section.

4.2.1 Word Modes

The architecture provides two main operating modes, *single* and *dual*, selectable at run time. In single mode, the unit operates with a 32-bit word size, whereas in dual mode it is configured for 16-bit words. In dual mode, each 32-bit word stored in RAM is partitioned into its upper and lower halves, which are treated as two independent 16-bit words, enabling parallel processing and improved throughput. Additionally, *poly* mode is implemented as a variant of dual-mode, with all operations, except modular multiplication, executed

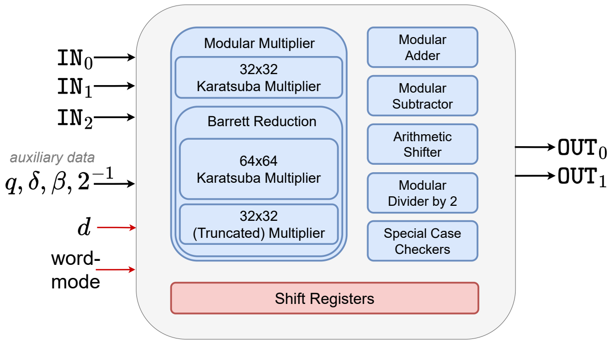


Figure 4: High-level architecture of the BU.

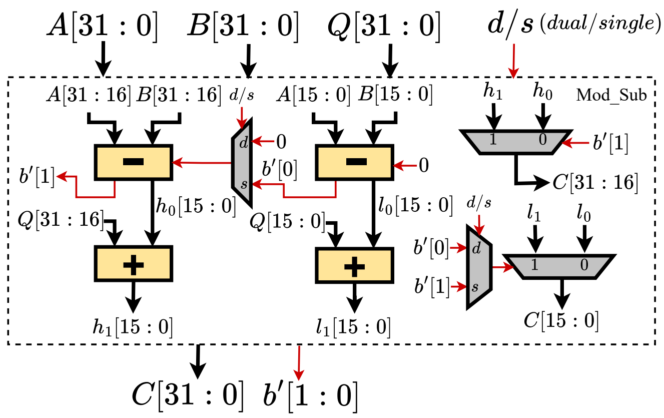


Figure 5: Modular subtractor architecture with dual-mode option. Black lines denote data paths, and red lines denote control and status signals.

identically. In this mode, the multiplication operation is realized as degree-1 polynomial multiplication, supporting modulus sizes of up to 16-bit. The poly-mode is beneficial for implementing the base multiplication in incomplete NTT, such as the case of Kyber. The pseudo-code for modular multiplication in poly mode is given as follows:

$$C_H = A_L \cdot B_H + A_L \cdot B_H \qquad \bmod q \tag{5}$$
$$C'_L = A_L \cdot B_L \qquad \bmod q \tag{6}$$
$$T = A_H \cdot B_H \qquad \text{(not reduced)} \tag{7}$$

An additional output T is produced, which serves as an intermediate result together with C'_L. To complete the base multiplication in incomplete NTT domain, T is multiplied by ζ in single mode and accumulated to C'_L, again in single mode:

$$C_L = C'_L + T \cdot \zeta \qquad \bmod q \tag{8}$$
$$= A_L \cdot B_L + A_H \cdot B_H \cdot \zeta \qquad \bmod q \tag{9}$$

Note that ζ is the twiddle factor, a power of the primitive root ψ (see Section 4), and it is distinct for the indices of the element-wise multiplication.

4.2.2 Butterfly Unit (BU)

The architecture of the BU is illustrated in Figure 4. The unit comprises several submodules: a modular adder, a modular subtractor, a modular multiplier, a module that performs modular division by 2, an arithmetic shifter to the right, and shift (delay) registers. All of the mentioned submodules are designed such that they support single and dual modes (and poly mode for the modular multiplier). BU operates in a pipelined manner. The shift registers are used to synchronize the intermediate data for each operation. It takes three 32-bit operands, and outputs two 32-bit results. The modulus q, δ, β, and 2^{-1} are auxiliary data (which are not pipelined) used in related operations, as detailed in the following paragraphs.

The modular adder computes the sum of two operands and, if the result exceeds the modulus q, subtracts q to ensure that the output remains within the valid range. The modular subtractor operates in an analogous manner, performing subtraction followed by addition when necessary. To illustrate dual-mode support, we present the architecture of the modular subtractor in Figure 5 as a reference, which is similar to the modular adder/subtractor architecture proposed in [AMI+23], but extended to support 16- and 32-bit operands. Other modules are implemented in a similar fashion. The borrow bits output, b', in the modular subtractor are used as inequality flags in several operations.

Modular Multiplier. The modular multiplier module consists of a 32-bit integer multiplier and a modular reduction unit. The modular reduction is implemented using Barrett reduction method [Bar87a]. The integer multiplier is responsible for implementing the poly mode and is realized using the Karatsuba multiplication algorithm [Kar63]. The design rational behind choosing Barrett reduction is to recover the quotient from the modular reduction operation as well. In other words, the modular multiplier unit is also used as a divider. Computing the quotient is important for supporting functions of PQC such as Compress in Kyber and Decompose in Dilithium. Another advantage is that, with Barrett reduction, the output of the operation is not multiplied with a factor as in Montgomery or Plantard. To support division by arbitrary 32-bit values, rather than restricting the dividend to a function of q^2, we implement the naïve Barrett reduction presented in Algorithm 1 (with $\lambda = 32$), which involves one 64-bit multiplication and one 32-bit multiplication. This is particularly needed when implementing a division needed in masked implementation of Kyber, as further discussed in Section 6.2. For efficient computation, the 64-bit multiplication is also implemented using a recursive Karatsuba algorithm. The pre-computed Barrett constant (denoted by δ in Figure 4) is supplied to the BU through the auxiliary data path. For the 32-bit multiplication in Barrett reduction, only the lower 32 bits of the product are required, resulting in a truncated multiplication. For both 32-bit and 64-bit Karatsuba multipliers, the core multiplication size is set to 16-bit, resulting in 3 and 9 core multiplications, respectively. As the truncated 32-bit multiplication is also achieved with 3 core multiplications of 16-bit, the modular multiplier requires 15 core multiplications in total. Without considering masking, the more efficient Barrett variant in Algorithm 2 could be used (again with $\lambda = 32$). Consequently, the Barrett reduction could be realized with only two 32-bit multiplications (one Karatsuba and one truncated multiplication), reducing the overall cost of modular multiplier to 9 core multiplications.

Next, we detail how related functionality for PQC are implemented by the BU architecture. Giving this unit its name, the butterfly circuits of the NTT and INTT are implemented within the BU, utilizing the modular multiplier, modular adder, and modular subtractor. Specifically, Cooley–Tukey [CT65] (CT) butterflies are used for NTT and Gentleman–Sande [GS66] (GS) butterflies are used for INTT. Note that INTT requires post-processing, where both operands must be multiplied by 2^{-1} after each GS butterfly. The post-processing for one operand is embedded into multiplication by ψ^{-1} performed in the GS butterfly. For the other operand, it is handled using the modular divider by 2 sub-module. This operation does not require multiplication and is implemented as $(x \gg 1) + 2^{-1} \mod q$ using the pre-calculated $(2^{-1} \mod q)$ value.

Compress, Decompress (Kyber): As shown in Eq. 1, Compress operation requires both multiplication and division with rounding. $2^d \times x$ is calculated in modular multiplier. Then, $q/2$ is added to the result using the modular adder, to simplify rounding using the adder. Finally, the division operation is done using Barrett Reduction. In Decompress (Eq. 2), the divisor 2^d is a power-of-two. Therefore, the division is implemented using the arithmetic shifter. These primitives are also used for performing left or right shifts, or divisions. Rounding can be disabled by a control bit. In particular, by setting $q = 1$, left and right shifts can be realized using Compress and Decompress, respectively. Similarly, invoking Compress with $d = 0$ is equivalent to computing the division x/q.

Decompose, UseHint (Dilithium): Similar to Compress, Decompose requires division by rounding (see Eq. 3), which is also efficiently done by the Barrett reduction in the modular multiplier, $r_1 = (x')/\alpha$, $r_0' = (x + \alpha_h) \mod \alpha$, where $x' = (x + \alpha_h)$ and $\alpha_h = (\alpha - 1)/2$. Note that r_1 and r_0' are the division and the remainder from dividing x' to α. The added constant is subtracted afterwards using the modular subtractor for computing the lower-order term, $r_0 = (r_0' - (\alpha_h)) \mod \alpha$. Moreover, there is a dedicated logic responsible for checking and applying the special case of the Decompose operation, abstracted as special case checkers in Figure 4. The corner case required for this check (by re-purposing 2^{-1} input), as well as α (setting $\beta = \alpha$), is given through the auxiliary data. We should note that the Decompose operation flow can be used to perform Power2Round by disabling the post-processing. UseHint operation is built on top of the Decompose output. BU receives both parts of the output produced by Decompose as its input, namely r_1, r_0, in addition to the hint h. The sign check on r_1 is reformulated as a norm check, $r_1 \leq q/2$, and performed as described in below paragraph. Based on the outcome of this check, r_1 is either incremented or decremented by 1 using the modular adder. We remark that the Decompose operation can be considered as a generic operation that returns both the division and remainder.

CheckNorm, MakeHint (Dilithium): The infinity norm check $||\mathbf{x}||_\infty \geq \beta$ over a vector requires performing an inequality check for every element in it, $x \geq \beta$. First, $\beta - 1$ is added to the input data utilizing the modular adder unit, $x' = x + \beta - 1$. As a result, the check becomes $x' \geq (2\beta - 1)$. Then, we perform $x'' = (2\beta - 2) - x'$ using the modular subtractor (x'' is the value before modular reduction). $x \geq \beta$ if and only if x'' is negative, so we output the borrow bits from the modular subtractor. MakeHint uses the exact same steps, with an additional step for checking the special case as in Decompose. The bound $\beta' = \beta - 1$ is also provided through the auxiliary data, (the same signal used in Decompose), as well as the special case for MakeHint. Although these mainly target Dilithium, inequality check is a generic feature.

4.2.3 Encode Unit (EU)

As discussed in Section 2.2, the decode function maps a sequence of bits to polynomial coefficients, while the encode function performs the inverse operation. In Kyber, where $\lceil \log q \rceil = 12$, there is no fixed alignment between the indices of 32-bit words and the 12-bit polynomial coefficients. The situation is the same for Dilithium as the modulus is 23 bits. As a result, explicit bit shifts are needed to transform between bit strings and polynomial coefficients. As in BU, EU operates based on the word mode, which enables it to handle the word sizes of both Kyber and Dilitihum efficiently. The internal resources are shared between encode and decode modes. EU is implemented as a shift-register. Unlike BU, the EU is a stateful unit. It maintains a 64-bit register K, implemented as a shift register, and a counter tracking the number of valid bits stored in K. We intentionally sized K to 64 bits (rather than 32) to provide buffering and prevent stalling in the input/output stream.

4.2.4 Sampling Unit (SU)

The sampling unit is responsible for two main pseudo-random sampling functions, CBD_η and rejection sampling. The former is used exclusively by Kyber, while the latter is a core building block in PQC, employed by both Kyber and Dilithium. For both operation modes, NTT-Lite takes a pseudo-random data string (possibly generated by an XOF) as input and applies the logic of these functions to that data. As shown in Figure 3, the SU takes its input from the EU, where the pseudo-random data is first decoded and split into chunks matching the bit-length processed by the SU. The SU contains two modular subtractors, and additional dedicated logic for both CBD_η and rejection sampling. The boundary for rejection sampling is supplied to NTT-Lite via the auxiliary data path. The SU also supports the dual-mode, consistent with the other units. We next describe the implementation details of these two sampling modes.

CBD_η takes $2 \times \eta$ input bits and outputs $(o_0 - o_1)$ where, $o_0 = (\sum_{i=0}^{\eta-1} b_i)$ and $o_1 = (\sum_{i=\eta}^{2\eta-1} b_i)$ with each b_i sampled uniformly from $\{0, 1\}$. Intuitively, o_0 and o_1 are the Hamming weights of two η-bit binary strings, and CBD_η computes the difference between these weights. In our implementation, EU decodes the pseudo-random input to data chunks of $2 \times \eta$ bits, each corresponding to the bits contributing to $(o_0||o_1)$, which are then processed by SU. In SU, each chunk is further decoded to o_0 and o_1 and the subtraction operation is performed.

The rejection sampling uses the first modular subtractor as an inequality checker, to decide $x < \beta$ by looking at the borrow bits. If the test passes, the data is forwarded to the output. In dual-mode, performing rejection sampling on 32-bit inputs requires a 16-bit buffer to handle scenarios in which only one of the two half-words is rejected. In some cases, the output of the rejection sampling must be centered, as in Dilithium's small-coefficient sampler, whose output is uniform in $[-\eta, \eta]$. To achieve this, we compute $\beta/2 - x$ for centralization. The second modular subtractor performs this operation.

4.3 High-Level Operations

In this section, we detail how the NTT-Lite operates on vector inputs leveraging the BU, EU, and SU. Table 3 summarizes the set of operations implemented in NTT-Lite. Although the table indicates the primary scheme associated with each operation, this association is not mandatory; the configuration is never hard-coded, ensuring crypto agility. The parameter n', which defines the input array length, is not fixed, and $\log n'$ can be configured to any positive integer up to 8.

4.3.1 Operation Details and Latencies

For `OP_NTT` and `OP_INTT`, there are $\log n'$ stages and $n'/2$ butterfly operations are performed by BU at each stage, leading to roughly $n'/2 \log n'$ clock cycles. We note that each operation incurs a small additional latency due to the pipelined BU, SU, and EU units, as well as from data read and write operations; these latencies are omitted in Table 3 as well as the rest of the discussion for clarity. For example, `OP_NTT` and `OP_INTT` indeed take $n/2 \cdot \log n + \mathcal{L}$ where $\mathcal{L}$ denotes the summation of butterfly and internal memory access latency which is 12 in our implementation.

`OP_SUM` computes the sum of all indices in an input of size n', utilizing the modular addition in BU, taking $n' - 1$ clock cycles. `OP_ADD`, `OP_SUB`, `OP_PWM` (point-wise multiplication), `OP_COMPRESS`, `OP_DECOMPRESS`, `OP_DECOMPOSE`, `OP_MAKE_HINT` are element-wise operations utilizing the corresponding operations from BU. Recall from Section 4.2.1 that `OP_PWM` in poly-mode is handled differently. Since a double pass over the input operands is performed to handle the multiplications by twiddle factors (see Eq. 8), the latency is approximately $2n'$ cycles. `OP_MAC` is similar to `OP_PWM`, while it operates in a multiply-

Table 3: Operations supported by NTT-Lite.

Opcode	**Source Type**	**Used By**	**Active Units**	**Clock Cycles**
OP_NTT	I	K, D	BU	$n'/2 \cdot \log n'$
OP_INTT	I	K, D	BU	$n'/2 \cdot \log n'$
OP_PWM	II	K, D	BU	n' / $2n'$
OP_ADD	II	K, D	BU	n'
OP_SUB	II	K, D	BU	n'
OP_SUM	III	K, D	BU	$n' - 1$
OP_MAC	VI	K, D	BU	n' / $2n'$
OP_COMPRESS	III	K	BU	n'
OP_DECOMPRESS	III	K, D[2]	BU	n'
OP_DECOMPOSE	V	D	BU	n'
OP_CHK_NORM	III	D	BU	$n'^{\star}$
OP_MAKE_HINT	III	D	BU	n'
OP_USE_HINT	VI	D	BU	n'
OP_ENCODE	III	K, D	EU	n'
OP_DECODE	IV	K, D	EU	n'
OP_CBD	IV	K	EU, SU	n'
OP_REJ_SAMP	IV	K, D	EU, SU	[1]

K: Kyber, D: Dilithium.
n' denotes the number of 32-bit words in the data array, which is $n = 256$ for Dilithium and $n/2 = 128$ for Kyber. * : operation is stopped immediately if operation returns failure, takes n' cycle if operation returns success.
[1] : Running time depends on the bound (β) and input data.
[2] : Used in masked Dilithium implementation, see Section 6.2.

and-accumulate manner, particularly beneficial for matrix multiplications. OP_CHK_NORM is also applied element-wise, but terminates early if any element x satisfies $|x| \geq \beta$. This variable-time behavior is acceptable for Dilithium, as the input is discarded and the process is restarted upon failure.

For OP_ENCODE, the n' input elements are streamed to the EU one-by-one, the operation completes once the final 32-bit encoded word is produced. Although the number of output words depends on the encoding parameter d, the total latency is determined by the input length n'. OP_DECODE and OP_CBD follow a similar pattern, the number of cycles is fixed by the n' output elements, while the required number of input words depends on d. For OP_REJ_SAMP, the user specifies the input length, and execution terminates once n' valid samples are generated; otherwise, all input is consumed. Thus, its runtime depends on the modulus, the input size, and the input values themselves.

4.3.2 Data Sources

The source and destination memories for each operation are classified under the *Source Type* column in Table 3. Type I operations require three input sources: two operands are fetched from the 128×32 RAMs and the third from the 256×32 RAM, such as OP_NTT and OP_INTT. In this case, the third operand corresponds to the twiddle factors. This class produces two outputs, written to the 128×32 RAMs. For the rest of the discussion, the concatenation of two 128×32 RAM is called RAM_0, and 256×32 RAM is called RAM_1. Type II operations require two operands, one is read from RAM_0 and the other is read from RAM_1. Type III operations require only a single input source, which is always read from the RAM_0. For Type IV such as OP_DECODE, the input is read from the RAM_1

to prevent overwriting its data prior to processing. For Types II, III, and IV, the output is written to the RAM_0. `OP_DECOMPOSE` produces two outputs: the primary output is written to the RAM_0, whereas the secondary output is stored in the RAM_1. This configuration is indicated as Type V. Finally, `OP_MAC` and `OP_USE_HINT` operations are classified under Type VI, as these have three source operands with a single output. In that case, while the first two operands come from RAM_0 and RAM_1, the third input source is the fresh data that is being transferred to NTT-Lite. That is directly processed without storing it internally.

In order to perform any of these opcodes provided by NTT-Lite, the input data must be transferred to NTT-Lite memory blocks RAM_0 and/or RAM_1, depending on the operation. While the data transfer seems as a performance drawback, in most operations, transferring the input to NTT-Lite memory, either to RAM_0 or RAM_1, and the operation is performed in parallel. Thanks to this overlapping mechanism, one of the transfer costs for the inputs is eliminated. Moreover, input and output data transfers are not always performed, enabling efficient *chained* operations on data already present in NTT-Lite. This means that NTT-Lite can be programmed to reuse one of the inputs or the output of a previous operation as the input to the next operation. This feature is not hard-coded and fully programmed by the user. All operation codes can be chained.

NTT-Lite also allows the right-hand operand to be a constant, supplied as auxiliary data, which is advantageous in scenarios where one input remains fixed. We leave increasing the size of NTT-Lite memory blocks for further optimization as a future work.

Lastly, NTT-Lite provides a dedicated flag to clear its internal memory at the end of an operation. This is beneficial for avoiding transitional leakage when two shares are processed by sequentially NTT-Lite. This cleaning occurs without additional cost, as it is overlapped with ongoing operations.

5 Algorithms for Masked Implementations

In order to protect Kyber and Dilithium implementations against differential power analysis attacks, all components that process the secret key need to be masked. This section describes how fully masked Kyber and Dilithium implementations are realized in the proposed HW/SW co-design solution (Section 3). To this end, we first introduce our approaches for masking the non-linear operations (CBD, norm checking, decompose, etc.), which are tweaked to leverage the existing hardware accelerators (NTT-Lite, Keccak and X2X) and maximize overall performance. Then, we provide masked compositions for high-level functions, Kyber.CCAKEM.Dec and Dilithium.Sign that build on these components.

5.1 Kyber

Here, we introduce the masked gadgets employed in our masked Kyber implementation and present the masked decapsulation algorithm Kyber.CCAKEM.MaskedDec.

5.1.1 Masked Binomial Sampling

The $\mathsf{MaskedCBD}_\eta$ gadget used in our work is presented in Algorithm 3. It follows the masked generation of a pseudo-random byte string from the sensitive r value in Lines 6-8 of Kyber.CCAKEM.Dec in Figure 11, using the Boolean masked PRF (Keccak). The $\mathsf{MaskedCBD}_\eta$ transforms uniformly distributed byte strings (input s in Algorithm 3) into polynomial coefficients (output z in Algorithm 3) that follow the centered binomial distribution over $[-\eta, \eta]$. Note that s refers to the 2η-bit substrings of the output of PRF, and $\mathsf{MaskedCBD}_\eta$ is applied to each substring to generate the required number of polynomial coefficients.

Algorithm 3 $\mathsf{MaskedCBD}(s^{\{0:d\}_B}, \eta)$ from [OSPG18, SPOG19]

Input: CBD parameter η
Input: Masking order d
Input: $s^{\{0:d\}_B} \in \mathbb{Z}_{2^{2\cdot\eta}}^{\{0:d\}_B}$
Output: $z^{\{0:d\}_A} \in \mathbb{Z}_q^{\{0:d\}_A}$ following CBD distribution

1: **for** $i = 0$ to $2 \cdot \eta - 1$ **do**
2: $\quad s'^{\{0:d\}_B}[i] = \mathsf{Refresh}(s^{\{0:d\}_B}[i])$ ▷ Using X2X
3: $\quad t^{\{0:d\}_A}[i] = \mathsf{B2A}_q(s'^{\{0:d\}_B}[i])$ ▷ Using X2X
4: **end for**
5: $z^{\{0:d\}_A} = 0$
6: **for** $i = 0$ to $\eta - 1$ **do**
7: $\quad z^{\{0:d\}_A} = z^{\{0:d\}_A} + t^{\{0:d\}_A}[i] - t^{\{0:d\}_A}[\eta + i] \mod q$ ▷ using NTT-Lite
8: **end for**
9: **return** $z^{\{0:d\}_A}$

Recall from Section 4.2.4 that CBD_η requires computing the differences of Hamming Weights of two η-bit strings. A first-order masked binomial sampler was proposed for the lattice-based NEWHOPE scheme in [OSPG18], and later extended in [SPOG19] to support arbitrary moduli q and masking orders d. In essence, this approach relies on first independently transforming each bit of the 2η-bit boolean shared string to arithmetic shares (using 2η calls to $\mathrm{B2A}_q$) to compute the difference between their Hamming weights, by subtracting and adding coefficient-wise in the arithmetic domain.

Further, the authors of [SPOG19] propose an alternate high-order masked binomial sampler, which performs the Hamming weight computation directly in the Boolean domain and only requires a single B2A conversion. In order to speed-up the Hamming weight computation and addition (in Boolean domain), the authors propose to use explicit bitslicing operations. As the proposed hardware architecture (Section 3) supports efficient mask conversion through the X2X accelerator, there is no need to include additional bitsliced arithmetic in the design (thus avoiding extra area overhead). Instead, we can rely on the existing units to perform multiple $\mathrm{B2A}_q$ operations efficiently.

To satisfy the required security notion, we introduce an explicit $\mathsf{Refresh}$ operation for the input s (Line 2).

We now show Algorithm 3 to be t-SNI secure with $t + 1$ shares, providing resistance against a probing adversary with t probes. Additionally, it allows the gadget to be used in larger compositions, such as a full Kyber decapsulation.

Theorem 1. *The gadget* $\mathsf{MaskedCBD}$*, presented in Algorithm 3, is t-SNI with $t \leq d$.*

Proof. The proof follows directly from the $\mathsf{Refresh}$ being t-SNI, as proposed in [SPOG19, Alg. 20], and the $\mathrm{B2A}_q$ being t-NI, as proven in [NDKV25]. Furthermore, the additions in Line 7 are performed share-wise, and thus can be modeled as a t-NI gadget. The $\mathsf{Refresh}$ operation stops the propagation of output/intermediate probes to the input s. □

5.1.2 Masked Compression with Subtraction

The $\mathsf{MaskedSubCompress}$ gadget, presented in Algorithm 4, allows to perform the masked compression used in Line 11 and Line 13 of $\mathsf{Kyber.CPAPKE.Enc}$ and Line 5 of $\mathsf{Kyber.CPAPKE.Dec}$ in Figure 11, and an optional subtraction. The latter is used to perform the final comparison in Line 5 of $\mathsf{Kyber.CCAKEM.Dec}$ in Figure 12, where the input b to Algorithm 4 corresponds to a compressed polynomial coefficient from the reference ciphertext c (repeated across all indices) in $\mathsf{Kyber.CCAKEM.Dec}$ in Figure 12. The masked compress operation is computed by performing the modulus switching using more precision, denoted by α. The input is arithmetically shared, which enables efficient additions (Lines 2-6) using the NTT-Lite, after which the shared data is converted to the Boolean domain using the A2B conversion (X2X accelerator), during which an (optional) mask refresh is performed (Line 7). The conversion is required to perform the final shift (Line 9) in the Boolean domain.

Algorithm 4 $\mathsf{MaskedSubCompress}(a^{\{0:d\}_A}, b, \delta)$ from [CGMZ23, DVBV22]

Input: $a^{\{0:d\}_A} \in \mathbb{Z}_q^{\{0:d\}_A}$
Input: $b \in \mathbb{Z}_{2^\delta}$, subtraction operand which is a compressed polynomial coefficient
Input: Compression factor δ
Output: $r^{\{0:d\}_B} \in \mathbb{Z}_{2^\delta}^{\{0:d\}_B}$ such that $r = \mathsf{Compress}_q(a, \delta) - b$

1: $\alpha = \lceil \log_2(q \cdot d) \rceil$
2: **for** $i = 0$ to d **do**
3: $\quad t^{\{i\}_A} = \lfloor (a^{\{i\}_A} \cdot 2^{\delta+\alpha} + q/2)/q \rfloor \mod 2^{\delta+\alpha+1}$ ▷ Using NTT-Lite
4: **end for**
5: $t^{\{0\}_A} = t^{\{0\}_A} + 2^{\alpha-1} \mod 2^{\delta+\alpha+1}$ ▷ Using NTT-Lite
6: $t^{\{0\}_A} = t^{\{0\}_A} - 2^{\alpha} \cdot b \mod 2^{\delta+\alpha+1}$ ▷ Using NTT-Lite
7: $t'^{\{0:d\}_A} = \mathsf{Refresh}(t^{\{0:d\}_A})$ ▷ Using X2X
8: $t^{\{0:d\}_B} = \mathsf{A2B}_{2^{\delta+\alpha+1}}(t'^{\{0:d\}_A})$ ▷ Using X2X
9: $r^{\{0:d\}_B} = (t^{\{0:d\}_B} \gg \alpha) \mod 2^\delta$ ▷ Using NTT-Lite
10: **return** $r^{\{0:d\}_B}$

Finally, we show that Algorithm 4 is t-SNI secure with $t+1$ shares, allowing it to be used in a larger composition.

Theorem 2. *The gadget* $\mathsf{MaskedSubCompress}$*, presented in Algorithm 4, is t-SNI secure with $t \leq d$.*

Proof. The t-SNI property follows directly from the Refresh (Line 7) being t-SNI [SPOG19, Alg. 20]. Otherwise, the algorithm consist of a series of t-NI operations: all additions and the logical shift are performed in a share-wise fashion and the A2B operation is t-NI, as proven in [NDKV25]. □

We note that, without the t-SNI Refresh in Line 7, the algorithm achieves the t-NI notion. During the full Kyber decapsulation, we will use both these variants (and security notions) of Algorithm 4.

5.1.3 Masked Non-Zero Testing

The final step in the Kyber decapsulation is the ciphertext comparison, in which the re-computed ciphertext c' is compared to the original ciphertext c (Line 5 of $\mathsf{Kyber.CCAKEM.Dec}$ in Figure 12). We present our proposed masked gadget for masked non-zero testing in Algorithm 5. Our methodology relies on testing the equality of two integer coefficients u and u' being equivalent to testing if $u - u'$ being zero or not, which is computed by $\mathsf{MaskedSubCompress}$ (Algorithm 4). Note that $\mathsf{MaskedSubCompress}$ is executed for each element of the input vectors $\mathbf{u}$ and $\mathbf{v}$, prior to invoking Algorithm 5, and the elements of these vectors indeed corresponds to the $\mathsf{MaskedSubCompress}$ outputs. The comparison is performed in the masked domain, and on all coefficients at once, as it was shown in [BDH+21] that partial results of the comparison leaks secret information. As such, the comparison is condensed into a single bit, which is unmasked ($\top$ or $\bot$).

Correctness. For the false positive analysis, we consider whether an element of $\mathbf{u}$ or $\mathbf{v}$ can be non-zero while the algorithm outputs $\bot$. This scenario can only occur if a modular reduction takes place during the summation. However, the carrier modulus q' is

Algorithm 5 $\mathsf{MaskedZeroTestVec}(\mathbf{u}^{\{0:d\}_B}, \mathbf{v}^{\{0:d\}_B})$

Input: $\mathbf{u}^{\{0:d\}_B} \in (\mathbb{Z}_{2^{\delta_u}}^{\{0:d\}_B})^{k \times n}$
Input: $\mathbf{v}^{\{0:d\}_B} \in (\mathbb{Z}_{2^{\delta_v}}^{\{0:d\}_B})^{n}$
Input: A carrier prime q' that satisfies $q' > k \cdot n \cdot (2^{\delta_u} - 1) + n \cdot (2^{\delta_v} - 1)$
Output: $\perp$ if any of the elements of $\mathbf{u}$ or v are non-zero. Otherwise, $\top$
1: $t^{\{0:d\}_A} \in \mathbb{Z}_{q'}^{\{0:d\}_A} = 0$
2: **for** $j = 0$ to $n-1$ **do**
3:     **for** $i = 0$ to $k-1$ **do**
4:         $\mathbf{u}^{\{0:d\}_A}[i][j] = \mathsf{B2A}_{q'}(\mathbf{u}^{\{0:d\}_B}[i][j])$ ▷ Using X2X
5:         $t^{\{0:d\}_A} = t^{\{0:d\}_A} + \mathbf{u}^{\{0:d\}_A}[i][j] \mod q'$ ▷ Using NTT-Lite
6:     **end for**
7:     $\mathbf{v}^{\{0:d\}_A}[j] = \mathsf{B2A}_{q'}(\mathbf{v}^{\{0:d\}_B}[j])$ ▷ Using X2X
8:     $t^{\{0:d\}_A} = t^{\{0:d\}_A} + \mathbf{v}^{\{0:d\}_A}[j] \mod q'$ ▷ Using NTT-Lite
9: **end for**
▷ Below lines correspond to [CGMZ23, Alg. 1]
10: **for** $i = 0$ to d **do**
11:     $r \leftarrow \mathbb{Z}_{q'}^{*}$ ▷ Using X2X
12:     $t^{\{0:d\}_A} = t^{\{0:d\}_A} \cdot r \mod q'$ ▷ Using NTT-Lite
13:     $t^{\{0:d\}_A} = \mathsf{Refresh}(\mathsf{t}^{\{0:d\}_A})$ ▷ Using X2X
14: **end for**
15: $t = t^{\{0\}} + t^{\{1\}} + \cdots + t^{\{d\}} \mod q'$ ▷ Unmasking, using NTT-Lite
16: **if** $t \neq 0$ **then return** $\top$
17: **else return** $\perp$
18: **end if**

chosen such that the accumulated sum never reaches the modulus boundary. Specifically, q' is selected so that even in the worst case, when all elements of $\mathbf{u}$ and $\mathbf{v}$ attain their maximum values, namely $(2^{\delta_u} - 1)$ and $(2^{\delta_v} - 1)$, the total sum remains strictly less than q'. Furthermore, the multiplication in Line 12 does not introduce a false positive, since $r \neq 0$ and q' is a prime, implying that $t^{\{0:d\}_A} \cdot r \equiv 0 \pmod{q'}$ if and only if $t^{\{0:d\}_A} = 0$. On the other hand, if all elements of $\mathbf{u}$ and $\mathbf{v}$ are zero, then $t^{\{0:d\}_A} = 0$, and the algorithm correctly outputs $\perp$, thereby eliminating false negatives.

Theorem 3. *The gadget* $\mathsf{MaskedZeroTestVec}$*, presented in Algorithm 5, is t-NI secure with $t \leq d$.*

Proof. It is shown in [CGMZ23] that Line 10-14 (without final unmasking) is t-NI secure. As the B2A$_q$ (Line 4 & 7) are t-NI and the additions in Line 5 & 8 are performed in each share domain, it follows trivially that the entire gadget is t-NI secure. □

5.1.4 Masked Decapsulation

Finally, all proposed gadgets are considered together to construct a fully masked Kyber decapsulation, $\mathsf{Kyber.CCAKEM.MaskedDec}$. For this, we consider all relevant steps in the algorithm in Figure 12 (and subroutines in Figure 11), except the operations involving public values (e.g. public key, ciphertext, etc.), and perform a fine-grained analysis of its composed security. A summary of the proposed construction of the masked Kyber decapsulation is depicted in Figure 6.

First, we analyze $\mathsf{Kyber.CPAPKE.Dec}$, as shown in Figure 11, and map its operations to gadgets as follows:

- G_{1-4}: Decode$_{12}$ (Line 2), $\bar{\mathbf{s}} \circ \mathsf{NTT}(\mathbf{u})$ (Line 5), INTT (Line 5), subtraction with v

(Line 5). All are linear (polynomial) operations and are performed on each share separately.

- G_5: Compress$_q$ (Line 5). We use the masked compression gadget in Algorithm 4 (t-NI), without the (optional) subtraction (b set to 0). We refer to this instantiation of Algorithm 4 by MaskedPolyToMsg.

Before the re-encryption in Kyber.CCAKEM.Dec function Figure 12 (Line 4), the decrypted message m' and the hash of the public key are hashed (using G hash function) to generate r and K' (mapped to gadget G_6), which are both sensitive. If the final ciphertext comparison is successful, the masked K' is unmasked to perform the final hash in the clear, as proposed in [BGR+21, BDK+21, KDVB+22]. It is critical that the short-term secret K' is only revealed if the comparison passes, and not in case of failure as then it could be used to recover the long-term secret key. The G operation, which relies on Keccak, is masked as proposed in prior art [GSM17] and integrated in our accelerator/peripheral.

Next, we analyze Kyber.CPAPKE.Enc (Figure 11), focusing on operations involving secret data:

- G_{7-9}: $\mathsf{PRF}(r)$ (Lines 5-8). This operation is also a symmetric primitive and implemented using the masked Keccak peripheral.

- G_{10-12}: CBD (Lines 5-8) to generate $\mathbf{r}, \mathbf{e}_1, e_2$. We utilize our proposed masked binomial sampler (Algorithm 3), which uses the X2X peripheral to accelerate the mask conversion.

- G_{13-16}: $\mathsf{NTT}(\mathbf{r})$ (Line 9), $\bar{\mathbf{A}} \circ \bar{\mathbf{r}}$ (Line 10), INTT (Line 10), addition with $\mathbf{e}_1$ (Line 10). All are linear operations and thus are masked by implementing them in a share-wise fashion.

- G_{17}: $\mathsf{Decompress_q}(\mathsf{Decode}_1((m'), 1))$ (Line 12). As before, the decoding operation is trivial to mask. Then, the Boolean shared message m' is converted to arithmetic shares with a $\mathrm{B2A}_q$ using the X2X peripheral, and multiplied with a constant. We refer to this gadget as MaskedPolyFromMsg.

- G_{18-21}: $\bar{\mathbf{t}} \circ \bar{\mathbf{r}}$, INTT, addition with e_2, addition with decompressed message (Line 12). All are linear operations and are performed on each share.

- G_{22-23}: $\mathsf{Compress}_q$ (Lines 11 & 13). The re-computed ciphertext components are compressed, and subtracted with the public (input) ciphertext $\mathbf{u}, v$, using the proposed MaskedSubCompress gadget (Algorithm 4). To compute the ciphertext component c_2, the (optional) explicit mask refresh in Algorithm 4 is enabled to ensure the composition is secure.

Finally, the masked ciphertext comparison is performed on the (masked) compressed ciphertext ($\mathbf{c}_1$ and c_2) using the secure zero check in Algorithm 4, mapped to G_{24}. As shown in Section 5.1.3, t is unmasked as a final step to determine if the comparison is successful (or not). Note that Figure 6 excludes the unmasking of t as well as short-term secret K'.

We now show that our proposed masking of the Kyber decapsulation is t-NI secure with $t+1$ shares, and as a result also t-probing secure. In the proof, we iterate over all output and intermediate variables and show how each can be simulated using only a limited set of input shares, based on the t-(S)NI properties of the individual gadgets.

Theorem 4. *The gadgets in Figure 6 (*Kyber.CCAKEM.Dec*) are t-NI secure with $t \leq d$.*

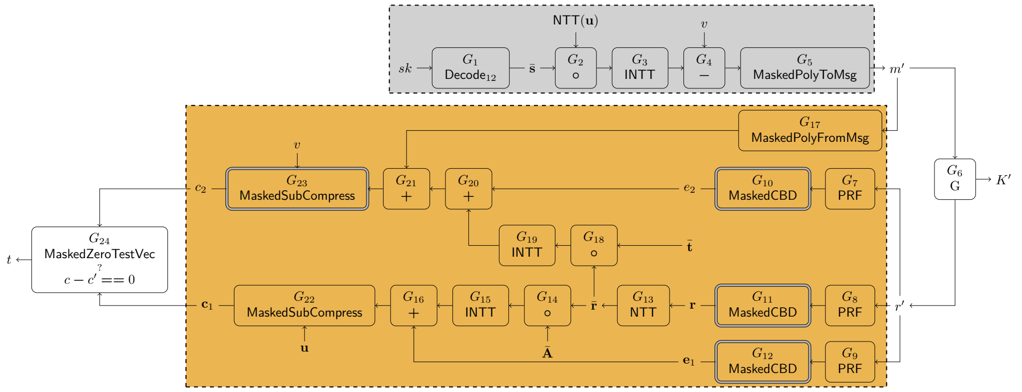


Figure 6: An abstract diagram of the Kyber.CCAKEM.MaskedDec, consisting of the decryption (gray) and re-encryption (yellow). The t-NI gadgets are depicted with a single border, the t-SNI gadgets with a double border.

Proof. All linear operations, in both the decryption and (re-) encryption, are modeled as t-NI gadgets as these are performed in a share-wise fashion (e.g. G_{1-4}, G_{13-16}, G_{18-21}). Additionally, all gadgets that perform symmetric key operations (G, PRF) are implemented as proposed in [GSM17] using t-NI DOM gates. As a result, the gadgets G_6 and G_{7-9} are modeled as t-NI gadgets. The Compress$_q$ algorithm is implemented using the proposed Algorithm 4, which is proven to be t-SNI secure (G_{23}) or t-NI (G_5, G_{22}). Similarly, the Decompress$_q$ operation consist of the t-NI B2A operation (X2X) and share-wise multiplication, and thus is modeled as the t-NI gadget G_{17}. The sampling operation (G_{10-12}) are implemented as the t-SNI gadget in Algorithm 3. The final comparison operation is implemented as proposed in Algorithm 5, and thus G_{24} is t-NI.

An adversary can probe any of these gadgets G_i internally (t_{G_i} probes) or their output shares (o_{G_i} probes), except the final outputs K' and t (due to t-NI property). In total, at most $t_{G_{\mathsf{Kyber.CCAKEM.Dec}}}$ internal probes can be placed:

$$t_{G_{\mathsf{Kyber.CCAKEM.Dec}}} = \sum_{i=1}^{24} t_{G_i} + \sum_{i=1}^{5} o_{G_i} + \sum_{i=7}^{23} o_{G_i}$$

The proof now consists of showing that any internal and output probes can be perfectly simulated with $\leq t_{G_{\mathsf{Kyber.CCAKEM.Dec}}}$ shares of the input sk, and that the required set of input shares $|I|$ is independent of $|O|$.

To simulate the internal and output probes in the gadgets of the decryption operation ($G_1 - G_5$, gray in Figure 6), a total of $t_{\text{gray}} = \sum_{i=1}^{5} t_{G_i} + o_{G_i}$ shares of the input sk are required due to the t-NI property of the gadgets.

For the encryption, we need to verify that the simulation of t_j probes on intermediate values does not require more than t_j shares of the inputs. Starting from the output and working towards the input, without the t-SNI gadget G_{23}, simulating $t_{G_{24}}$ intermediate probes of G_{24} would require $\geq 2 \cdot t_{G_{24}}$ shares of $\bar{\mathbf{r}}$, which is not secure. But, the t-SNI gadget stops the propagation of probes on the output shares ($o_{G_{23}}$) to its input, so that $t_{G_{23}} + o_{G_{23}} + t_{G_{24}}$ probes can be simulated with $t_{G_{23}}$ shares of the output of G_{21}. In summary, to simulate the internal probes and output shares of gadgets $G_{14} - G_{16}, G_{18} - G_{24}$ require $t_{\bar{\mathbf{r}}} = (\sum_{i=14}^{16} t_{G_i} + o_{G_i}) + (\sum_{i=18}^{22} t_{G_i} + o_{G_i}) + t_{G_{23}} + t_{G_{24}}$ shares of $\bar{\mathbf{r}}$. Similarly, we can determine the amount of shares of the intermediate values in the encryption (yellow) are required for simulation:

$$t_{e_2} = t_{G_{20}} + o_{G_{20}} + t_{G_{21}} + o_{G_{21}} + t_{G_{23}}$$

$$t_{\mathbf{e}_1} = t_{G_{16}} + o_{G_{16}} + t_{G_{22}} + o_{G_{22}} + t_{G_{24}}$$

We now repeat the same technique as before and determine the total amount of shares of the inputs (m' and r') of the yellow zone (encryption) are required to simulate all probes in the yellow-shaded zone. We sum all shares, starting from the output and working towards the input:

$$\begin{aligned} t_{\text{yellow}} = t_{G_7} + o_{G_7} + t_{G_8} + o_{G_8} + t_{G_9} + o_{G_9} + t_{G_{10}} + t_{G_{11}} \\ + t_{G_{12}} + t_{G_{17}} + o_{G_{17}} + t_{G_{21}} + o_{G_{21}} + t_{G_{23}} \end{aligned}$$

As we can see, the t-SNI property of $G_{10} - G_{12}$ and G_{23} stops the propagation of probes.

Finally, we sum all share contributions from the yellow and gray zone, to determine the set of input shares required for simulating the entire composition:

$$|I| = t_{\text{gray}} + t_{G_6} + t_{\text{yellow}}$$

As $|I| \leq t_{G_{\mathsf{Kyber.CCAKEM.Dec}}}$, the composed gadget satisfies the t-NI notion. □

5.2 Dilithium

Here, we describe the masked gadgets employed in our masked Dilithium implementation and present the masked signing algorithm Dilithium.MaskedSign.

5.2.1 Masked Uniform Sampler

In the masked Dilithium.Sign algorithm, the masked uniform sampler, here referred to as MaskedUniform in Algorithm 6, is utilized to mask the function ExpandMask and generate $\mathbf{y}$ securely (Line 7, Dilithium.Sign algorithm, Figure 13). MaskedUniform takes as input the Boolean masked string s and the distribution parameter γ. Note that the string s indeed corresponds to substrings of the output of XOF, which takes as input the secret seed ρ' in Line 7 of Dilithium.Sign. Algorithm 6 is then executed across all substrings to generate the polynomial coefficients of $\mathbf{y}$. Each share of the input string s is of γ bits. This masked algorithm outputs arithmetic masked r, where each coefficient of each share belongs to $\mathbb{Z}_q$ and each coefficient of r follows the uniform distribution in $[-2^{\gamma-1}+1,\ 2^{\gamma-1}]$.

Algorithm 6 $\mathsf{MaskedUniform}(s^{\{0:d\}_B}, \gamma)$

Input: Parameter γ
Input: $s^{\{0:d\}_B} \in \mathbb{Z}_{2^\gamma}^{\{0:d\}_B}$
Output: $r^{\{0:d\}_A} \in \mathbb{Z}_q^{\{0:d\}_A}$, where r follows uniform distribution in $(-2^{\gamma-1}, 2^{\gamma-1}]$
1: $s^{\{0:d\}_B} = \mathsf{Refresh}(s^{\{0:d\}_B})$ ▷ Using X2X
2: $r^{\{0:d\}_A} = \mathsf{B2A}_q(s^{\{0:d\}_B})$ ▷ Using X2X
3: $r^{\{0\}_A} = 2^{\gamma-1} - r^{\{0\}_A} \mod q$ ▷ Using NTT-Lite
4: **for** $i = 1$ to d **do**
5: $\quad r^{\{i\}_A} = 0 - r^{\{i\}_A} \mod q$ ▷ Using NTT-Lite
6: **end for**
7: **return** $r^{\{0:d\}_A}$

Theorem 5. *The gadget* MaskedUniform*, presented in Algorithm 6, is t-SNI with $t \leq d$.*

Proof. It follows an identical approach to the proof of Theorem 1. Refresh is a t-SNI gadget and B2A$_q$ is a t-NI gadget. The subtractions in Lines 3 and 5 are performed in a share-wise manner, preserving the t-NI security. Therefore, the gadget MaskedUniform, a composition of t-SNI and t-NI gadgets, also achieves t-SNI security. □

5.2.2 Masked Norm Check

The masked norm check algorithm, referred to as $\mathsf{MaskedCheckNorm}_w$ in Algorithm 7, is used to securely perform both rejection checks $||\mathbf{z}||_\infty \geq \gamma_1 - \beta$ and $||\mathbf{r}_0||_\infty \geq \gamma_2 - \beta$ (Line 14, Dilithium.Sign algorithm, Figure 13). $\mathsf{MaskedCheckNorm}_w$ takes arithmetic shares of a, the word size parameter w, and the bound β as input. It returns a masked 1-bit output r, which is 0 if the following equation holds: $q - \beta \geq a \geq \beta$ and otherwise 1. Our $\mathsf{MaskedCheckNorm}_w$ is adapted from [ABC+23].

Algorithm 7 $\mathsf{MaskedCheckNorm}_\omega(a, \beta)$ adapted from [ABC+23]

Input: Bound β
Input: Word-size parameter $\omega \geq \lceil \log q \rceil + 1$, *e.g.*, $\omega = 32$
Input: $a^{\{0:d\}_A} \in \mathbb{Z}_q^{\{0:d\}_A}$
Output: $r^{\{0:d\}_B} \in \mathcal{B}_1^{\{0:d\}_B}$ such that $r = 0$ if $(q - \beta \geq a \geq \beta)$. Otherwise, $r = 1$

1: $a^{\{0\}_A} = a^{\{0\}_A} + \beta - 1 \mod q$ ▷ Using NTT-Lite, the check becomes $(a \geq 2\beta - 1)$ ▷ Modulus switching
2: $a^{\{0:d\}_B} = \mathsf{A2B}_q(a^{\{0:d\}_A})$ ▷ Using X2X
3: $t^{\{0:d\}_A} \in \mathbb{Z}_{2^\omega}^{\{0:d\}_A} = \mathsf{B2A}_{2^\omega}(a^{\{0:d\}_B})$ ▷ Using X2X
4: $t^{\{0\}_A} = t^{\{0\}_A} - (2\beta - 1) \mod 2^\omega$ ▷ Using NTT-Lite, the check becomes $(t \geq 0)$
5: $t^{\{0:d\}_B} = \mathsf{A2B}_{2^\omega}(t^{\{0:d\}_A})$ ▷ Using X2X
6: $r^{\{0:d\}_B} = t^{\{0:d\}_B}[\omega - 1]$ ▷ Using NTT-Lite, logical shift to right by $\omega - 1$-bit
7: **return** $r^{\{0:d\}_B}$

Now, we show that Algorithm 7 is t-NI secure.

Theorem 6. *The gadget* $\mathsf{MaskedCheckNorm}_w$*, presented in Algorithm 7, is t-NI with $t \leq d$.*

Proof. In Algorithm 7, the $\mathrm{B2A}_q$ on Line 2, the $\mathrm{B2A}_{2^w}$ on Line 3, and the $\mathrm{A2B}_{2^w}$ on Line 5 are t-NI. The addition in Line 1, the subtraction in Line 4, and the logical shift operation in Line 6 are performed in a share-wise manner, thus preserving the t-NI security. Therefore, the gadget $\mathsf{MaskedCheckNorm}_w$, a composition of t-NI gadgets, also achieves t-NI security overall. □

5.2.3 Masked Decompose Function

The masked decompose function, MaskedDecompose in Algorithm 8, is used to extract high and low bits of $w = (W_1, W_0)$ securely (Lines 9 and 13, Dilithium.Sign algorithm, Figure 13). MaskedDecompose takes arithmetic shares of a and a decomposition parameter γ as input and returns unmasked high bits r_1 and arithmetic masked shares of low bits r_0, where unmasked $a = r_1 \cdot \delta + r_0$ and $\delta = (q-1)/\gamma$. Our MaskedDecompose closely follows SecDecomposeComp from [ABC+23]. As proven in [ABC+23], the MaskedDecompose in Algorithm 8 is t-NI secure when r_1 is the public output.

5.2.4 Masked Signing

In this section, we describe our masked signing operation for Dilithium, Dilithium.MaskedSign. For our masked signing algorithm, we follow the sensitive analysis proposed in [ABC+23] and mask the same variables. All the gadgets required to mask Dilithium.Sign operation (Figure 13) are presented in Figure 7. For the sake of simplicity, we omit the component from the figure that only handles public or non-sensitive values, and for which masking is unnecessary.

Now, we elaborate on the gadgets utilized in Dilithium.MaskedSign and align them with Dilithium.Sign operation presented in Figure 13.

Algorithm 8 $\mathsf{MaskedDecompose}(a^{\{0:d\}_A}, \gamma)$ from [CGTZ23]

Input: $a^{\{0:d\}_A} \in \mathbb{Z}_q^{\{0:d\}_A}$
Input: Decomposition parameter γ
Output: $r_1, r_0^{\{0:d\}_A}$ such that $(r_1, r_0) = \mathsf{Decompose}_q(a, \gamma)$

$\delta = (q-1)/\gamma$
$t^{\{0:d\}_A} = -a^{\{0:d\}_A} \cdot \delta \mod q$ ▷ Using NTT-Lite
$t^{\{0\}_A} = t^{\{0\}_A} + (q-1)/2 \mod q$ ▷ Using NTT-Lite
$t^{\{0:d\}_B} = \mathsf{A2B}_q(t^{\{0:d\}_A})$ ▷ Using X2X
if δ is a power-of-two **then**
  $r_1^{\{0:d\}_B} = t^{\{0:d\}_B}[\log \delta - 1 : 0]$
  $r_1 = r_1^{\{0\}_B} \oplus r_1^{\{1\}_B} \oplus \cdots \oplus r_1^{\{d\}_B}$ ▷ Unmasking
else
  $t^{\{0:d\}_B} = \mathsf{Refresh}(t^{\{0:d\}_B})$ ▷ Using X2X
  $r_1^{\{0:d\}_A} = \mathsf{B2A}_\delta(t^{\{0:d\}_B})$ ▷ Using X2X
  $r_1 = r_1^{\{0\}_A} + r_1^{\{1\}_A} + \cdots + r_1^{\{d\}_A} \mod \delta$ ▷ Unmasking, using NTT-Lite
end if
$r_0^{\{0:d\}_A} = a^{\{0:d\}_A} - \gamma \cdot r_1 \mod q$ ▷ Using NTT-Lite
return $r_1, r_0^{\{0:d\}_A}$

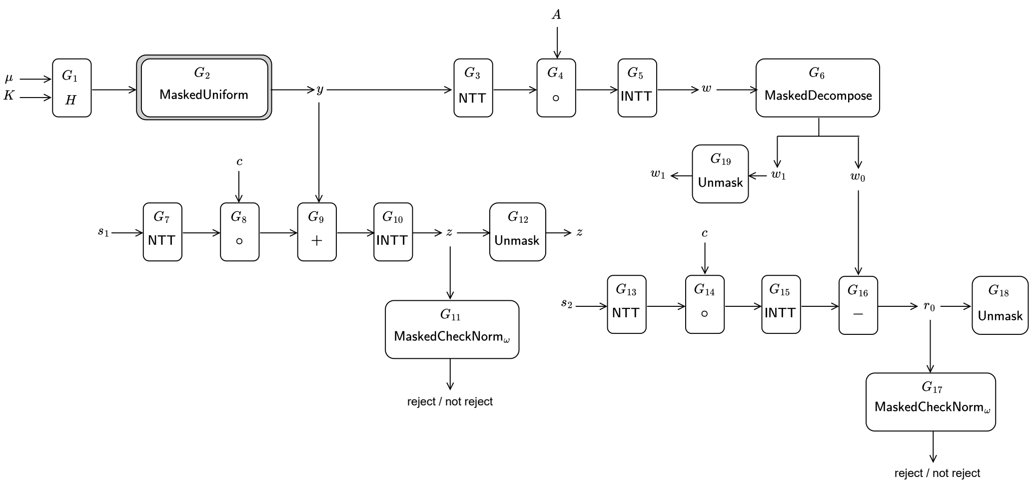


Figure 7: An abstract diagram of the $\mathsf{Dilithium.MaskedSign}$. All the gadgets are at least t-NI. The t-NI gadgets are depicted with a single border, while the t-SNI gadget $\mathsf{MaskedUniform}$ is depicted with a double border.

- G_{1-2}: As part of the $\mathsf{ExpandMask}$ function (Line 7), masked Keccak is used as the gadget G_1 first, and after that $\mathsf{MaskedUniform}$ (Algorithm 6) is used as the gadget G_2 to sample $\mathbf{y}$ from $[-2^{\gamma-1}+1,\ 2^{\gamma-1}]$ following the uniform distribution.
- G_{3-5}: $\mathsf{NTT}(y)$, $\mathsf{NTT}(y) \circ A$, and $\mathsf{INTT}(\mathsf{NTT}(y) \circ A)$ of Line 8 are all linear operations and are performed securely by applying them share-wise. The masked $\mathsf{NTT}(y)$, masked $\mathsf{NTT}(y) \circ A$, and masked $\mathsf{INTT}(\mathsf{NTT}(y) \circ A)$ are performed using G_3, G_4, and G_5 gadgets, respectively.
- G_6: The decomposition function generates unmasked w_1 used in $\mathsf{HighBits}_q(\mathbf{w}, 2\gamma_2)$ (Line 9) and the arithmetic shares of w_0 used in $\mathsf{LowBits}_q(\mathbf{w} - c\mathbf{s_2}, 2\gamma_2)$ (Line 13). It

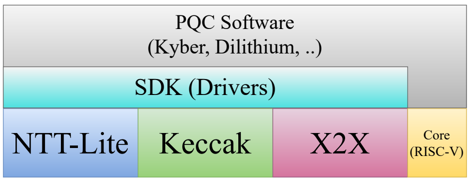


Figure 8: Software development stack for RISQrypt.

is performed using the MaskedDecompose function in Algorithm 8 as the gadget G_6.

- G_{7-10}: $\mathsf{NTT}(\mathbf{s_1})$, $\mathsf{NTT}(c) \circ \mathsf{NTT}(\mathbf{s_1})$, $\mathsf{NTT}(\mathbf{y}) + (\mathsf{NTT}(c) \circ \mathsf{NTT}(\mathbf{s_1}))$, and $\mathsf{INTT}(\mathsf{NTT}(\mathbf{y}) + (\mathsf{NTT}(c) \circ \mathsf{NTT}(\mathbf{s_1})))$ carried out according to the operations presented in Line 12, which are all linear and can be masked by applying them share-wise. These are implemented using G_7, G_8, G_9, and G_{10} gadgets, respectively.
- G_{13-16}: Similarly, $\mathsf{NTT}(\mathbf{s_2})$, $\mathsf{NTT}(c) \circ \mathsf{NTT}(\mathbf{s_2})$, $\mathsf{INTT}(\mathsf{NTT}(c) \circ \mathsf{NTT}(\mathbf{s_2})$, and $\mathbf{w} - c\mathbf{s_2}$ (on low bits) are performed corresponding to the operations specified in Line 13. These are also linear and can be masked by applying them share-wise. These masked operations are carried out with gadgets G_{13}, G_{14}, G_{15}, and G_{16}, respectively.
- G_{11} and G_{17}: The gadget $\mathsf{MaskedCheckNorm}_\omega$, presented in Algorithm 7 is used as G_{11} and G_{17}. G_{11} and G_{17} are used to perform the first two rejection checks $||\mathbf{z}||_\infty \geq \gamma_1 - \beta$ and $||\mathbf{r}_0||_\infty \geq \gamma_2 - \beta$ on Line 14 securely.
- G_{12}, G_{18}, and G_{19}: These gadgets are used to unmask $\mathbf{z}$, $\mathbf{r}_0$, and w_1 securely.

Similar to the masked signing algorithm of Dilithium presented in [ABC+23], all gadgets of our masked signing algorithm are at least t-NI, and public variables are securely unmasked. Therefore, the masked signing algorithm achieves t-NI security, given the signature $\sigma = (\mathbf{z},\ \mathbf{h}, \bar{c})$ is a public output.

6 Software Layer

In this section, we describe the software layer of the RISQrypt PQC implementations. The overall software stack is illustrated in Figure 8. The SDK denotes the driver layer that interfaces with the RISQrypt hardware accelerators, namely NTT-Lite, Keccak, and X2X. In our design, the software primarily serves an orchestration role, issuing commands to the hardware accelerators and providing memory address pointers for input and output data.

6.1 Unprotected Implementations

The unprotected implementations cover all high-level APIs of Kyber and Dilithium (see Figure 12 and Figure 13). Recall that the integrated hardware accelerators support nearly all underlying subroutines: hashing and extendable-output functions are handled by the Keccak accelerator, while polynomial arithmetic and related operations are executed by NTT-Lite using the operation codes described in Section 4. The only exception is the SampleInBall function in Dilithium, which is not computationally intensive and has therefore been left for future work.

6.2 Protected Implementations

We now discuss the implementation of the software layers for the masked functions of Kyber decapsulation and Dilithium signature generation, Kyber.CCAKem.MaskedDec and

Dilithium.MaskedSign. The polynomial arithmetic in these functions, which is linear under arithmetic masking, is implemented in NTT-Lite via share-wise processing. Between the processing of different shares, the memory-clearing feature of NTT-Lite is utilized (see Section 4.3.2). Similarly, hashing operations such as SHA or SHAKE, with or without masking, are executed by the Keccak accelerator, which supports first-order masking. The remaining parts of the algorithms consist of non-linear operations, as discussed in the previous section. We now discuss how each of these operations maps efficiently to NTT-Lite and X2X, enabled by the agile design of the architecture.

First, we discuss the non-linear algorithms used in Kyber. In Algorithm 3, Refresh and $\mathsf{B2A}_q$ in Lines 2 and 3 are performed in a single command by the X2X accelerator (`OP_REFX2X`), thereby avoiding redundant processing in the pipeline. This operation also leverages the 1-bit mode of X2X. The arithmetic in Line 7 consists of classical chained additions and subtractions implemented in NTT-Lite. For the rest of the non-linear algorithms, the constant operand feature of NTT-Lite is applied where relevant, although it is not explicitly mentioned in the discussion.

In Algorithm 4, a division by q is required in Line 3, which is efficiently performed via Barrett reduction in NTT-Lite. It is worth noting that existing implementations using reduction mechanisms specific to Kyber's q cannot compute the quotient. Furthermore, this algorithm requires 32-bit (single-mode) processing because the dividend exceeds 16 bits, despite Kyber's general dual-mode operation. Switching between single-mode and half-mode is handled efficiently using `OP_ENCODE` and `OP_DECODE`, without involving software-level data processing. The shift in Line 9 is achieved via `OP_DECOMPRESS`.

In Algorithm 5, the summations in Lines 5 and 8 are carried out using `OP_ADD` or array-wise addition, followed by a final `OP_SUM` to perform accumulation across all indices. The carrier prime q' is supported by NTT-Lite thanks to its configurability. Similarly, X2X allows the use of this modulus because it does not restrict the supported moduli to those of Kyber and Dilithium. Random sampling in Line 11 and Refresh in Line 13 are performed using the X2X accelerator, using `OP_PRNG` and `OP_REF`, respectively. The MaskedPolyFromMsg algorithm (see G_{17} in Section 5.1.4) also leverages the 1-bit mode of X2X to perform the B2A efficiently on 1-bit input.

Next, we discuss the non-linear algorithms used in Dilithium. Algorithm 6 is straightforward to implement using the NTT-Lite and X2X accelerators. Algorithm 7 similarly relies on the standard functionality of these units. Notably, this algorithm operates modulo 2^{32} in NTT-Lite for the subtraction in Line 4. In Line 6, the right-shift is implemented using `OP_DECOMPRESS`, demonstrating the generality of this command, although it was originally designed for Kyber. In Algorithm 8, Line 9, it is again worth noting that X2X operates using moduli different from the default ones of Kyber and Dilithium. The XOR ($\oplus$) operation required by boolean unmasking in Line 7 is currently not implemented in NTT-Lite. However, it is trivial to add, and performing it in software does not introduce micro-architectural leakage concerns, as the data is no longer security-sensitive.

7 Results

In this section, we evaluate the proposed solution in terms of security and performance, and compare it with existing works in the literature. All source code related to the RISQrypt project is publicly accessible at `https://github.com/cisec-su/RISQrypt`. This includes the HDL sources regarding the hardware accelerators, the associated SDKs, software layers for PQC implementations, and SCA scripts including trace acquisition and testing.

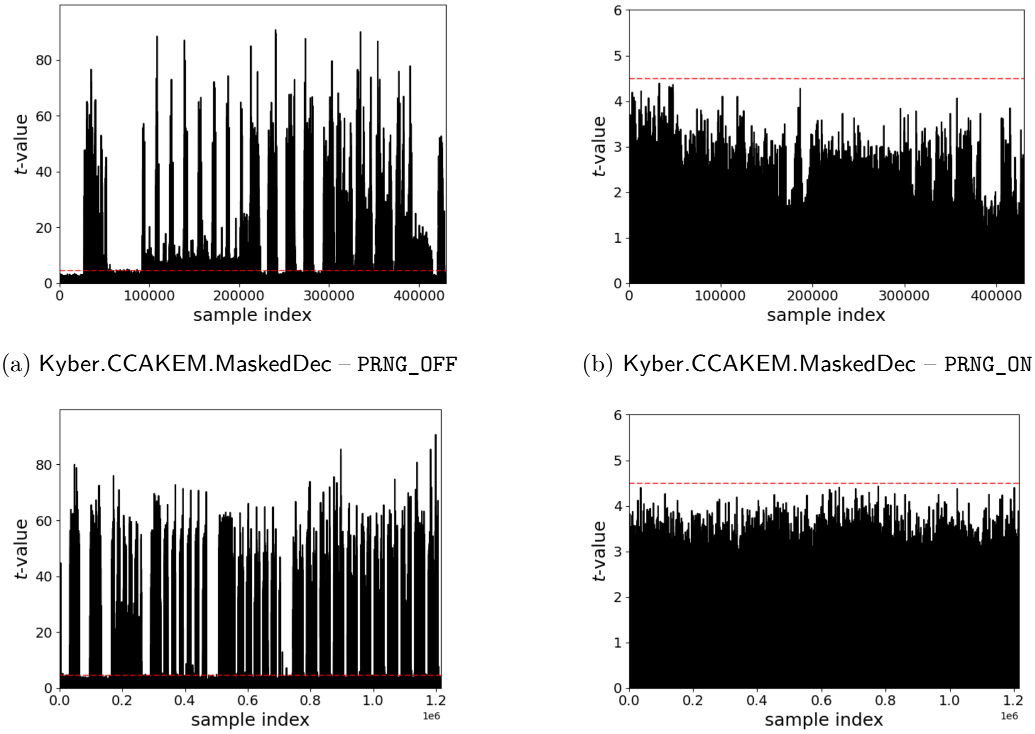
7.1 Side-channel Leakage Evaluation

Although we give a formal security proof for our side-channel-protected design, a practical evaluation is still necessary to uncover implementation issues such as transitional leakage. To this end, we performed a Test Vector Leakage Assessment (TVLA), a standard procedure widely used in the side-channel analysis literature [GGJR+11]. In TVLA, two sets of traces are captured from the evaluated implementation. A trace is a set of power leakages recorded during the execution of cryptographic function. One of the sets is captured with a random input, while the other is captured with a fixed input. The masked implementation is approved if the two trace sets are statistically not different from each other, which evidences that the power leakage does not depend on secret data. Welch's t-test is used as the statistical tool to compare the random-input and the fixed-input set, which computes a metric called t-value. The greater the t-value, the more different both sets are from each other. In the test, the null hypothesis proposes that the fixed-input and random-input trace sets follow the same statistics. A typical threshold for t-value is 4.5 in absolute value, which provides 99.999% confidence to reject the null hypothesis.

We used the ChipWhisperer framework for the experiments. CW Husky[2] is used for trace capturing while CW305[3] target board, which is equipped with an Artix-7 FPGA (XC7A100T), is used as the victim device running our implementation. The core clock of the victim device is set to 40 MHz. The trace collector facility is configured to collect four samples at each clock cycle. To validate our experimental setup, we also include the t-test results where the PRNG in the system is disabled. For this scenario, we collected 1K traces, whereas 100K traces were used when PRNG was enabled. We captured the fixed-input and random-input sets by randomly interleaving them to mitigate environmental bias [TG16].

We evaluated both Kyber.CCAKem.MaskedDec and Dilithium.MaskedSign. In our analysis, the input refers to the secret key used by the mentioned functions. For Dilithium.MaskedSign, the t-test is complex due to the algorithm being non-constant time (e.g., rejection sampling). We therefore analyze only a single iteration of the rejection sampling loop and ignore rejections within that iteration. Figure 9 demonstrates the t-test results for both algorithms. With the randomness disabled (PRNG OFF), leakage is clearly observed after acquiring only 1K traces, as t-values are significantly higher than 4.5. On the other hand, when the PRNG is on, the t-values remain lower than 4.5, for both algorithms. This result proves the practical effectiveness of countermeasures implemented. We note that since our t-test analysis is performed using a large number of samples, false positives (Type I error) are expected [DZD+17, BGG+14]. In [BGR+21], the authors use a more conservative threshold of 6.88 to reduce the probability of Type I error. Instead, we repeated the tests over the same intervals and consider effects that were not reproducible as false positives.

[2] https://rtfm.newae.com/Capture/ChipWhisperer-Husky/
[3] https://rtfm.newae.com/Targets/CW305%20Artix%20FPGA/



(a) Kyber.CCAKEM.MaskedDec – `PRNG_OFF`

(b) Kyber.CCAKEM.MaskedDec – `PRNG_ON`

(c) Dilithium.MaskedSign – `PRNG_OFF`

(d) Dilithium.MaskedSign – `PRNG_ON`

Figure 9: TVLA results under two configurations: `PRNG_OFF` and `PRNG_ON`.

7.2 Performance Evaluation

To assess the effectiveness of the proposed hardware–software co-design, we conducted a comprehensive performance evaluation and compared it with existing implementations. By benchmarking against representative designs, we highlight the advantages of our approach in achieving lower latency while maintaining competitive hardware costs.

7.2.1 Hardware Overhead

In Table 4, we report the resource overhead of cryptographic accelerators in the proposed design. We implemented our design on the Artix-7 XC7A100T-1CSG324C FPGA device using Xilinx Vivado 2023.2 with the balanced option. In the table, we report the FPGA resource consumption of three configurations. The first one is the base processor. Cryptographic accelerators are excluded, and the system contains only the RISC-V processor, memory, and peripherals. In the second one, cryptographic accelerators are included, but masking is disabled as a reference. The X2X accelerator is absent, and the Keccak accelerator does not support masking. This configuration serves as a performance and area reference. The third one is our proposed design, where cryptographic accelerators are included with full masking support. By comparing configurations with cryptographic acceleration, we quantify the overhead introduced by acceleration, both with and without masking. The reported results align with our design expectations. Overall, the reported hardware overhead represents a reasonable cost for achieving both security and high performance. The maximum frequency of the system decreases modestly when cryptographic acceleration is enabled. However, timing analysis shows that the critical path lies on the Wishbone bus. This is due to the increased overall design size and a

Table 4: FPGA resource overhead of cryptographic modules and masking.

Configuration: Crypto. Acc.	Configuration: Mask.	LUT	FF	DSP	BRAM	Freq. (MHz)
✗	NA	5488	2703	0	80	63
✓	✗	15672	8126	9*	81.5	61
Crypto Overhead		10184	5423	9*	1.5	
✓	✓	30074	21460	15	81.5	60
Crypto Overhead		24586	18757	15	1.5	

* modular multiplier is not optimized for the unmasked scenario and theoretical number is reported. Maximum synthesisable frequencies are reported.

more constrained place-and-route process, rather than within the internal logic of the cryptographic accelerators.

Table 5 presents a detailed breakdown of resource utilization for individual modules. For NTT-Lite, the BU consumes the majority of LUTs and all DSPs, as it implements the core polynomial arithmetic. The cost of EU and SU are relatively negligible. For Keccak and X2X, the core units also consume the largest portion of resources. In Keccak, FIFO and FSM also consume notably. The sponge function logic could be implemented in software to reduce FSM cost at the expense of performance (see Section 3.1). For reference, Table 6 shows the resource utilization of Keccak when masking is disabled, which approximately halves its resource consumption. This accounts for the observed overhead in LUT and FF usage. Accordingly, the cost of the modular multiplier in NTT-Lite decreases by 6 DSPs when masking is not considered (see Section 4.2.2). In X2X, the PRNG also consumes a significant number of LUTs; however, A2B/B2A conversion methods with lower randomness requirements could reduce this overhead, which we leave for future work.

Next, we present the results of ASIC synthesis for the proposed design in Table 7. The design is synthesized using the TSMC 65nm Low-Power (LP) standard cell library. We evaluated the design at two target frequencies: 100 MHz and 500 MHz. The latter incurs an area overhead of approximately 6% relative to the 100 MHz baseline, resulting from the timing-driven optimizations required to meet the stricter timing constraints. Notably, NTT-Lite consumes significantly more area than the others, primarily due to its memory (BRAM-equivalent) requirements. Further area optimization of NTT-Lite for ASIC implementations is left as future work.

7.2.2 Time Performance

Table 8 presents the time performance of our implementation in comparison with related work from the literature. Developed source codes for RISQrypt's software layer is compiled with RISC-V toolchain using `-O3` option to speed optimize. Reported numbers are in thousands of clock cycles. For our solution, the mean number of clock cycles is reported as the average over 1024 experiments with uniformly random inputs. For the existing works, the reported figures are taken directly from the corresponding publications. For Kyber, the performance of CCAKEM functions are reported. Input message length is set to 32B for Dilithium.

As can be observed in the table, our design significantly outperforms the literature across all functions of Kyber and Dilithium. For Kyber.INDCCA.MaskedDec, there is only

Table 5: FPGA resource utilization of individual modules

Module	LUT	FF	DSP	BRAM
NTT-Lite	5064	1934	15	1.5
BU	2761	1092	15	0
EU	331	73	0	0
SU	413	99	0	0
FSM + RAM	656	273	0	1.5
Keccak	9777	7626	0	0
Core	5512	4022	0	0
FIFO	1024	3200	0	0
FSM	3024	39	0	0
X2X	9409	9163	0	0
Core	3804	5084	0	0
PRNG	3341	1090	0	0
FSM + Reg. File	1225	1265	0	0

Table 6: FPGA resource utilization of Keccak when masking is disabled.

Module	LUT	FF	DSP	BRAM
Keccak	4792	3455	0	0
Core	2355	1617	0	0
FIFO	416	1600	0	0
FSM	1900	38	0	0

Table 7: ASIC synthesis results of cryptographic modules.

Module	Area (μm^2) @ 100 MHz	Area (μm^2) @ 500 MHz
NTT-Lite	271938	304825
Keccak	131513	131939
X2X	123122	125883
Crypto total	526573	562647

Table 8: Time performance comparison with prior work.

Work	Kyber768				Dilithium3			
	KeyGen	Enc	Dec	MaskedDec	KeyGen	Sign	Verify	MaskedSign
RISQrypt	**23**	**28**	**35**	**109**	**120**	**480**	**126**	**1230**
[FVR+22]	214	298	313	1235	–	–	–	–
[WZZ+24]	536	671	639	–	2087	5003	1986	–
[DMMM23]	663	856	1083	–	–	–	–	–
[AOP+25]	62	72	83	–	234	476	216	–
[NDMZ+21]	695[1]	732[1]	862[1]	–	2975	10212	2964	–
[MCL+23]	–	–	–	–	264[2]	546[2]	281[2]	–
[MBB+23]	–	–	–	–	479	3988	511	–
[KSFS24]	–	–	–	–	1068	3253	1127	–

Reported numbers are in thousands of clock cycles. [1] Based on INDCPA timing. [2] Estimated based on the reported clock frequency.

one prior work suitable for comparison, namely [FVR+22]. This study is the closest to ours in terms of scope and also reports the best performance among existing literature. The authors propose instruction set extensions for Keccak and mask conversions and a loosely coupled accelerator for polynomial arithmetic. However, since their mask conversion engine and Keccak accelerator are implemented as instruction set extensions, their flexibility and overall efficiency are inherently limited. As a result, our implementation outperforms their reported Kyber.INDCCA.MaskedDec performance by a factor of $\mathbf{11.3}\times$. A similar performance advantage is observed for the unmasked functions. In particular, for Kyber.INDCCA.Keygen, Kyber.INDCCA.Enc, and Kyber.INDCCA.Dec, our implementation achieves speed-ups of $\mathbf{9.3}\times$, $\mathbf{10.64}\times$, and $\mathbf{8.94}\times$, respectively. On the other hand, compared to [AOP+25], which reports the closest timing results, our implementation still achieves speed-ups of $\mathbf{2.69}\times$, $\mathbf{2.57}\times$, and $\mathbf{2.37}\times$ for Kyber.INDCCA.Keygen, Kyber.INDCCA.Enc, and Kyber.INDCCA.Dec, respectively.

For Dilithium.MaskedSign, there is **no directly comparable work in the literature**. Although [WZZ+24] also considers masking in their implementation, the applied countermeasure is limited to polynomial arithmetic. Therefore, their reported results for masked functions are not directly comparable to ours. Nevertheless, we compare our performance against unprotected implementations reported in the literature. Compared to [AOP+25], our implementation achieves speedups of $\mathbf{1.95}\times$, and $\mathbf{1.71}\times$, for Dilithium.Keygen, and Dilithium.Verify, respectively, while performance of Dilithium.Sign remains comparable. Note that the acceleration strategy of [AOP+25] differs from ours in that they adopt instruction set extensions on a RISC-V–based vector processor with high area overhead, which operates as a co-processor loosely coupled to the main processor. In the next section, we further show that our design achieves significantly better area efficiency than [AOP+25], in addition to the improved time performance. Another competitive implementation is [MCL+23][4], against which we achieve speedups of $\mathbf{2.19}\times$, $\mathbf{1.14}\times$, and $\mathbf{2.23}\times$, for Dilithium.Keygen, Dilithium.Sign, and Dilithium.Verify. We note that [MCL+23] focuses exclusively on Dilithium and does not provide a masked implementation. However, despite the narrower scope, their acceleration strategy is conceptually similar to ours, as they also employ loosely coupled accelerators for both NTT and Keccak while leaving a small portion of the computation to software.

In general, the primary reason for the superiority of our work is that, with the exception of [AOP+25, MCL+23], none of the existing studies employ loosely coupled accelerators for all required primitives. The works [NDMZ+21] and [MBB+23] focus on tightly coupled accelerators and consequently achieve only a limited level of acceleration. Although [WZZ+24] does not rely on tightly coupled acceleration, it does not include a hardware accelerator for Keccak and instead applies software-level optimizations. In [KSFS24], similar to [FVR+22], the authors employ a tightly coupled accelerator for Keccak while using a loosely coupled accelerator for the NTT. As a result, their performance remains below ours. Furthermore, [DMMM23] accelerates only Keccak. Overall, our superior performance demonstrates the advantage of employing loosely coupled accelerators for all primary operations.

In Figure 10, we analyze the time share of each accelerator in the system. Again, mean clock cycle values are reported over 1024 experiments with uniformly random inputs. Observe that NTT-Lite dominates the running time for all functions. This offers an opportunity to further speed-up the design by integrating more computation resources, i.e., multiple BUs, into NTT-Lite, which we leave for future work. It also confirms our design goal of supporting all arithmetic operations within NTT-Lite, in both masked and unmasked functions. The second most time-consuming accelerator for masked functions is X2X. In contrast, Keccak accounts for only a small fraction of the total execution time in our implementation, unlike the hashing bottlenecks reported in prior software-oriented

[4] The authors of [MCL+23] do not report clock cycle counts directly. Instead, they provide execution times in milliseconds, the operating frequencies of the target platforms (666 MHz for hardware and 150 MHz for software) and the ratio of hardware-accelerated subroutines as 0.94. We estimate a combined effective frequency as $176 \approx 666 \cdot (1 - 0.94) + 150 \cdot 0.94$ and compute the corresponding clock cycles based on the reported timings and this estimated frequency.

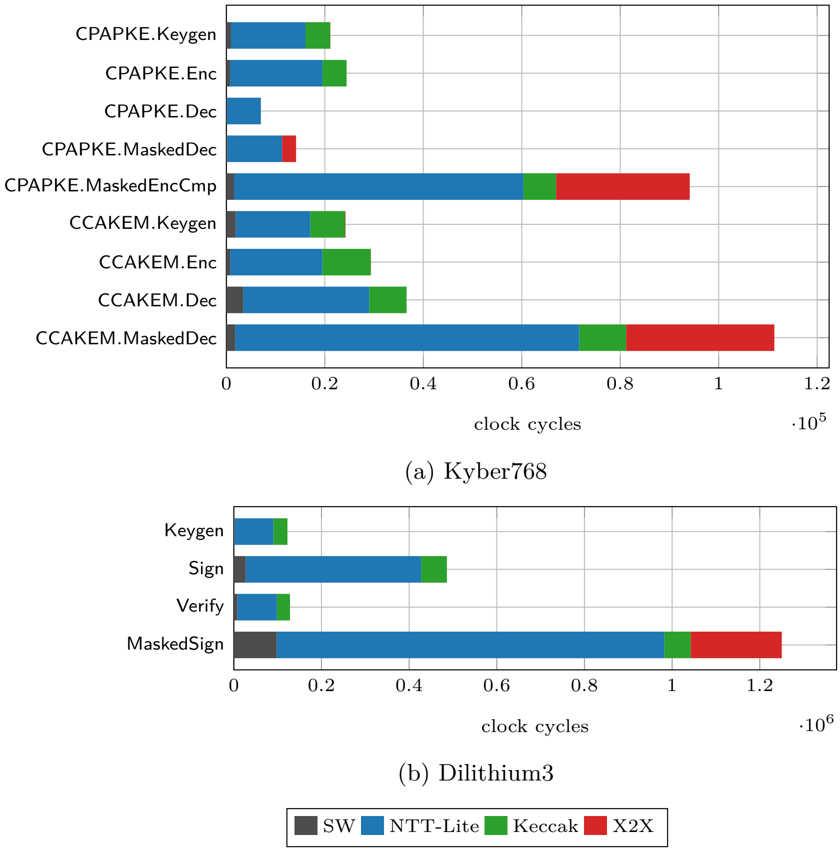


Figure 10: Number of clock cycles spent by accelerators.

studies [KRSS19]. The software execution contributes less than 8% of the total cycle count across all evaluated functions, highlighting the effectiveness and functional coverage of the proposed hardware acceleration. Particularly, it takes roughly 2% of the total cycles for Kyber.INDCCA.MaskedDec and 8% of the total cycles for Dilithium.MaskedSign. The ratio of mask conversion operations done by X2X in Kyber.INDCCA.MaskedDec is greater compared to Dilithium.MaskedSign, amounting to 27% and 16%, respectively. In Dilithium.MaskedSign, NTT-Lite accounts for 70% of the operations.

To further study the efficiency of our design for masked implementations, we report the time performance of the non-linear algorithms in Table 9 for both Kyber and Dilithium. Except for MaskedZeroTestVec, the algorithms were presented in single-word form in Section 5, while the time performance reported here corresponds to the execution of these algorithms on the entire polynomial in $\mathcal{R}_q$. Moreover, we report the time cost of pseudo-random sampling performed by Keccak accelerator, denoted by XOF. This applies to MaskedCBD and MaskedUniform, as these process the pseudo-random data produced (see Figure 6 and Figure 7). Observe that all of the utilized non-linear gadgets are realized highly efficiently, spending a few thousand clock cycles. Interestingly, the overhead of XOF to MaskedCBD is very small, around %4. We note that in XOF + MaskedCBD, the Keccak overhead is limited to the FIFO read operation, as the block permutation is executed in parallel with the MaskedCBD logic. For MaskedUniform used in Dilithium, the overhead of Keccak is around %30, since Keccak processes multiple blocks in this case and therefore must wait for the completion of the block permutation until all the blocks are squeezed. Although MaskedSubCompress and MaskedPolyToMsg are based on the same algorithm (Algorithm 4), the first takes more time as it requires 32-bit arithmetic.

Table 9: Time performance of non-linear algorithms in clock cycles.

Kyber768		**Dilithium3**	
MaskedCBD	3733	MaskedUniform	3639
XOF + MaskedCBD	3894	XOF + MaskedUniform	4714
MaskedSubCompress	5443	MaskedCheckNormω	6720
MaskedPolyFromMsg	1177	MaskedDecompose	7632
MaskedPolyToMsg	2037		
MaskedZeroTestVec	8765		

Table 10: Code size comparison with prior work.

Kyber768			**Dilithium3**		
Work	**Masking**	**Size (B)**	**Work**	**Masking**	**Size (B)**
RISQrypt	✓	17612	**RISQrypt**	✓	18848
[FVR+22]	✓	28554			
RISQrypt	✗	10236	**RISQrypt**	✗	14324
[FVR+22]	✗	13028	[WZZ+24]	✗	32120*
[WZZ+24]	✗	32520*	[KSFS24]	✗	20052
[AOP+25]	✗	11800	[AOP+25]	✗	21008

* Reported for Kyber1024 and Dilithium5; no significant difference is expected with respect to the security level.

Code size is another crucial consideration in embedded systems. In Table 10, we report the code size of our implementation and compare it with existing works from the literature. We present the code sizes separately for the masked and unmasked configurations, as different source files are compiled depending on the scenario. For example, in the masked configuration for Kyber, the compiled implementation includes Kyber.INDCCA.Keygen, Kyber.INDCCA.Enc, and Kyber.INDCCA.MaskedDec, whereas the unmasked configuration includes Kyber.INDCCA.Keygen, Kyber.INDCCA.Enc, and Kyber.INDCCA.Dec. The same approach is followed for Dilithium. The results show that our design is also competitive in terms of code size compared to prior work. The primary reason for this advantage is the reduced software workload, as also evidenced by the profiling results.

7.2.3 Area-Time Comparison

In this section, we perform an area-time evaluation of our architecture and demonstrate that the area overhead is well justified given the achieved performance improvements. Computing the Area–Time Product (ATP) for HW/SW co-designs is non-trivial; therefore, we adopt the following methodology. First, we estimate the hardware area overhead using $\text{Area}_{\text{HW}} = \text{LUT} + \text{FF}/2 + \text{DSP} \cdot 309 + \text{BRAM} \cdot 564$, which expresses all resources in LUT-equivalent units. The coefficients 309 and 564 were obtained by forcing Vivado to synthesize a 16×16-bit multiplier and a 1024×32 memory block using LUTs only, as previously done [PHGCGP22]. The multiplier size matches the core multiplication width of our architecture, while the memory configuration corresponds to the maximum depth of a single BRAM in Artix-7 with 32-bit data width. Since we consider HW/SW co-designs, we also include the area cost of the processor core for the software stack (e.g., RISC-V). We employ the area of the HORNET core used in RISQrypt, as reported in Table 4, excluding BRAM usage, for all designs in this section. Specifically, $\text{Area}_{\text{Core}} = \text{LUT}_{\text{Core}} + \text{FF}_{\text{Core}}/2 = 6839.5$. We then estimate the area overhead of the software by converting the reported code size into an equivalent number of BRAM blocks as $\text{Area}_{\text{SW}} = \text{BRAM}_{\text{Code}} = \text{Code Size}/4096$. The division by 4096 captures the byte capacity of a single 1024×32 BRAM block. The total area is then computed as $\text{Area}_{\text{T}} = \text{Area}_{\text{HW}} + \text{Area}_{\text{Core}} + \text{Area}_{\text{SW}}$ and the Area–Time

Table 11: Hardware utilization of our work compared to designs from the literature. Only the overhead of cryptographic modules are considered.

Work	Plat-form	Archi-tecture	LUT/FF/DSP/BRAM	$Area_{HW}$	Sch-eme	$Area_{SW/T}$	ATP
Masked							
RISQrypt	▲	L	24586/18757/15/1.5	39.45	K D	2.42/48.71 2.6/48.88	11.2/13.63/53.09 58.65/601.22/61.58
[FVR+22]	▲	L, T	16979/8834/7/20.5	35.12	K	3.97/45.93	98.3/136.88/567.28
Unprotected							
RISQrypt	▲	L	10184/5423/9/1.5	16.52	K D	1.4/24.77 1.97/25.33	5.7/6.94/8.67 30.40/121.6/31.92
[FVR+22]	▲	L, T	7687/3515/7/4.5	14.15	K	1.79/22.78	48.75/67.88/71.3
[WZZ+24]	▼	L	3087/1050/4/3 6639/1660/16/3	6.54 14.11	K D	4.59/17.97 4.57/25.52	96.33/120.59/114.84 532.55/1276.63/506.77
[DMMM23]	▲	L	7128/3798/0/0	9.02	K	NA/18.42	122.16/157.73/199.56
[NDMZ+21]	◆	T	178/0/5/0.5 377/0/10/0.5	2.01 3.75	K D	NA/11.4 NA/14.28	79.26/83.48/98.31 424.71/1457.86/423.14
[MBB+23]	▼	T	459/0/2/0	1.08	D	NA/11.6	55.58/462.77/59.3
[MCL+23]	▼	L	13128/11556/4/14	28.04	D	NA/37.93	100.13/207.10/106.58
[KSFS24]	■	L, T	7219/3238/7/6	14.43	D	2.76/24.03	256.65/781.71/270.82
[AOP+25]	▲	L	48158/16641/112/0	91.08	K D	1.62/99.55 2.89/100.81	61.72/71.67/82.62 235.91/479.89/217.77

HW/SW co-design approaches: (L) Loosely coupled, (T) Tightly coupled. Platforms: (▲) Artix7, (▼) Zynq700, (◆) ZCU106, (■) Zynq Ultrascale+. ✢ Limited SCA protection. ATP: KeyGen/Enc/(Masked)Dec (K) or KeyGen/(Masked)Sign/Verify (D). $Area_{SW/T}$: $Area_{SW}$ / $Area_{T}$.

Product is defined as ATP = (Area × Time). We exclude the clock frequency from the comparison, as it strongly depends on the target platform. For example, both our design and [FVR+22] use a soft RISC-V core on an Artix-7 FPGA, which results in relatively low operating frequencies; both implementations report clock frequencies of around 60 MHz. On the other hand, platforms such as Zynq-7000, which integrate an ARM processor, can reach up to 150 MHz, as reported in [MCL+23]. A fully fair comparison would require porting all designs to a common platform and re-evaluating them under identical conditions; however, such an effort would be engineering-intensive and beyond the scope of this work. Moreover, our design can be synthesized at 500 MHz clock frequency in ASIC environment (see Table 7), which is sufficient for the clock frequencies typically used in low-power microcontrollers. We underline that our goal is not to rank prior works against one another, but rather to justify the hardware overhead of our design by demonstrating that it achieves superior ATP.

For the comparison, certain missing data points had to be estimated, as not all components required for the ATP computation are consistently reported in the literature. In particular, the studies [NDMZ+21, MCL+23, MBB+23, DMMM23] do not provide code size information. For [NDMZ+21] and [MBB+23], we used the code size of a Dilithium implementation compiled with the RISC-V toolchain, as reported in [KSFS24].

For [MCL+23], we computed a code size by taking the average of the reported code sizes of works that support Dilithium and employ loosely coupled acceleration approach, namely our implementation, [WZZ+24], and [KSFS24]. Similarly for [DMMM23] and [NDMZ+21], we took the average of the Kyber code size reported by us, [FVR+22], and [WZZ+24]. These decisions do not affect the conclusions drawn in this section, as our implementation has the smallest code size among the works that report it (see Table 10), and code size is a minor factor overall.

Table 11 presents the computed ATP results of our work compared to the related designs from the literature. The results show that RISQrypt provides superior ATP results compared to the existing work. Although our design is slightly larger in several cases, its high performance compensates for the area overhead, resulting in a favorable overall cost. [FVR+22] utilizes 31% fewer LUTs and 53% fewer FFs compared to our design. However, due to its significantly higher BRAM consumption, the total area estimates (Area_T) of our design are only 1.06× and 1.08× larger for the masked and unprotected configurations, respectively. Recall that the authors of [FVR+22] propose tightly coupled accelerators for Keccak and mask conversion. Despite their slightly lower area estimate, our solution achieves approximately $\mathbf{8.22}\times$ to $\mathbf{10.68}\times$ better ATP results, due to its significantly improved time performance. This further underscores the advantage of our loosely coupled accelerator architecture across all operations. In the unprotected setting, the tightly coupled designs of [MBB+23, NDMZ+21] also stand out due to their very low area overhead. However, despite their smaller area footprint, the performance gains achieved by our approach more than compensate, resulting in superior ATP overall.

Recall that [AOP+25] leverages vector instructions in a loosely coupled co-processor for polynomial arithmetic. The high level of parallelism achieved through these vector instructions comes at a substantial area cost. Accelerator overhead of their design ($\mathrm{Area}_\mathrm{HW}$) is $\mathbf{5.52}\times$ larger than ours. This area advantage, combined with the time-performance improvements discussed in the previous section, results in $\mathbf{3.94}\times$ to $\mathbf{10.82}\times$ better ATP values.

The total area results of [WZZ+24] for Dilithium are very close to ours, while for Kyber their design consumes approximately $\mathbf{0.72}\times$ of the total area of our design. However, these results correspond to two separate, scheme-specific architectures, whereas our measurements are based on a unified architecture supporting both Kyber and Dilithium. When their two dedicated designs are deployed together (Area_T columns aggregated for Kyber and Dilithium) to provide support for both schemes, the total area cost becomes approximately $\mathbf{1.44}\times$ that of ours. Although the total area overhead of [KSFS24] is slightly lower than ours, our design achieves a $\mathbf{6.42}\times$ to $\mathbf{8.44}\times$ improvement in ATP. Area estimate of [MCL+23] is $\mathbf{1.5}\times$ that of our design, while supporting only Dilithium. Moreover, their modular reduction unit is specialized to the specific structure of Dilithium's modulus q, which limits generality. Specifically, our design achieves a $\mathbf{1.7}\times$ to $\mathbf{3.33}\times$ better ATP compared to [MCL+23].

8 Conclusion

We presented RISQrypt, a HW/SW co-design framework for secure, fast, and agile implementations of PQC. Our architecture integrates three dedicated hardware accelerators: NTT-Lite for polynomial arithmetic, Keccak, and X2X for A2B&B2A mask conversions. NTT-Lite is an efficient design, which is run-time configurable and optimized for both half- and full-word lengths needed by Kyber and Dilithium. We showed that all subroutines in Kyber and Dilithium are supported by the functionality of integrated hardware accelerators. Most importantly, we presented efficient implementations for both Kyber's masked decapsulation and Dilithium's masked signature generation, and demonstrated that they are side-channel secure both theoretically and in practice. We also discussed that the required building blocks for the masked implementations are also fully supported by X2X and NTT-Lite, avoiding data processing in the software core, which could otherwise introduce microarchitectural leakage. We showed that having a dedicated accelerator for each major computational primitive is essential to achieving high performance. As a result, our implementation achieves an order-of-magnitude speedup over the state of the art while also delivering a superior area–time product performance.

An additional advantage of employing a programmable polynomial arithmetic accelerator is the resulting agility of the design. Our hardware platform can be extended to support other algorithms built on similar primitives, such as Falcon [FHK+18] and SPHINCS+ [BHH+15] from the PQC standards. Moreover, it enables efficient and side-channel-secure implementations of client-side functions in Fully Homomorphic Encryption (FHE) schemes, including encryption and decryption for BFV [Bra12, FV12] and CKKS [CKKS17], as well as the transciphering algorithm PASTA [DGH+23]. We leave the detailed implementation and evaluation of these additional algorithms for future work.

9 Acknowledgments

This work is partially supported by the European Union's Horizon Europe research and innovation program under grant agreement No: 101079319, TUBITAK under Grant Number 122E222, the Horizon 2020 ERC Advanced Grant (101020005 Belfort) and the CyberSecurity Research Flanders with reference number VOEWICS02.

Appendices

A Kyber and Dilithium Algorithm Definitions

Algorithm specifications of Kyber and Dilithium are presented in Figure 11,Figure 12, and Figure 13.

$\mathsf{Kyber.CPAPKE.KeyGen}()$ ....

1. $d \leftarrow \mathcal{B}^{32}$
2. $(\rho, \sigma) = \mathsf{G}(d)$
3. **For** $i = 0$ to $k-1$
4.   **For** $j = 0$ to $k-1$
5. $\bar{\boldsymbol{A}}[i][j] = \mathsf{Parse}(\mathsf{XOF}(\rho, j, i))$
6. **For** $i = 0$ to $k-1$
7. $\mathbf{s}[i] = \mathsf{CBD}_{\eta_1}(\mathsf{PRF}(\sigma, i))$
8. $\mathbf{e}[i] = \mathsf{CBD}_{\eta_1}(\mathsf{PRF}(\sigma, i+k))$
9. $\bar{\mathbf{s}} = \mathsf{NTT}(\mathbf{s})$
10. $\bar{\mathbf{e}} = \mathsf{NTT}(\mathbf{e})$
11. $\bar{\mathbf{t}} = \bar{\mathbf{A}} \circ \bar{\mathbf{s}} + \bar{\mathbf{e}}$
12. $pk = \mathsf{Encode}_{12}(\bar{\mathbf{t}}||\rho)$
13. $sk = \mathsf{Encode}_{12}(\bar{\mathbf{s}})$
14. **return** (pk, sk)

$\mathsf{Kyber.CPAPKE.Enc}(pk, m, r)$ ....

1. $(\bar{\mathbf{t}}||\rho) = \mathsf{Decode}_{12}(pk)$
2. **For** $i = 0$ to $k-1$
3.   **For** $j = 0$ to $k-1$
4. $\bar{\boldsymbol{A}}[i][j] = \mathsf{Parse}(\mathsf{XOF}(\rho, j, i))$
5. **For** $i = 0$ to $k-1$
6. $\mathbf{r}[i] = \mathsf{CBD}_{\eta_1}(\mathsf{PRF}(r, i))$
7. $\mathbf{e}_1[i] = \mathsf{CBD}_{\eta_2}(\mathsf{PRF}(r, i+k))$
8. $e_2[i] = \mathsf{CBD}_{\eta_2}(\mathsf{PRF}(r, i+2k))$
9. $\bar{\mathbf{r}} = \mathsf{NTT}(\mathbf{r})$
10. $\mathbf{u} = \mathsf{INTT}(\bar{\mathbf{A}} \circ \bar{\mathbf{r}}) + \mathbf{e_1}$
11. $\mathbf{c}_1 = \mathsf{Compress}_q(\mathbf{u}, d_u)$
12. $v = \mathsf{INTT}(\bar{\mathbf{t}} \circ \bar{\mathbf{r}}) + e_2 + \mathsf{Decompress}_q(\mathsf{Decode}_1(m), 1)$
13. $c_2 = \mathsf{Compress}_q(v, d_v)$
14. **return** $c = (\mathbf{c_1}||c_2)$

$\mathsf{Kyber.CPAPKE.Dec}(sk, c)$ ....

1. $(c_1||c_2) = \mathsf{Decode}_{12}(c)$
2. $\bar{\mathbf{s}} = \mathsf{Decode}_{12}(sk)$
3. $\mathbf{u} = \mathsf{Decompress}_q(c_1, d_u)$
4. $v = \mathsf{Decompress}_q(c_2, d_v)$
5. $m = \mathsf{Compress}_q(v - \mathsf{INTT}(\bar{\mathbf{s}} \circ \mathsf{NTT}(\mathbf{u})), 1)$
6. **return** m

Figure 11: Kyber.CPAPKE algorithms from [BDK+22].

Kyber.CCAKEM.KeyGen ....

1. $z \leftarrow \mathcal{B}^{32}$
2. $(pk, sk') = \mathsf{Kyber.CPAPKE.KeyGen}(\lambda)$
3. $sk = (sk'||z)$
4. **return** (pk, sk)

Kyber.CCAKEM.Enc(pk) ....

1. $m \leftarrow \mathcal{B}^{32}$
2. $m = \mathrm{H}(m)$
3. $(K', r) = \mathsf{G}(m||\mathrm{H}(pk))$
4. $c = \mathsf{Kyber.CPAPKE.Enc}(pk, m, r)$
5. $K = \mathsf{KDF}(K'||\mathrm{H}(c))$
6. **return** (c, K)

Kyber.CCAKEM.Dec(c, sk, pk) ....

1. $(sk'||z) = sk$
2. $m' = \mathsf{Kyber.CPAPKE.Dec}(sk', c)$
3. $(K', r) = \mathsf{G}(m'||\mathrm{H}(pk))$
4. $c' = \mathsf{Kyber.CPAPKE.Enc}(pk, m', r)$
5. **If** $c = c'$
6. **return** $K = \mathsf{KDF}(K'||\mathrm{H}(c))$
7. **Else**
8. **return** $K = \mathsf{KDF}(z||\mathrm{H}(c))$

Figure 12: Kyber.CCAKEM algorithms from [BDK+22].

$\mathsf{Dilithium.KeyGen}(\lambda)$ ....

1. $\zeta \leftarrow \mathcal{B}^{32}$
2. $(\rho, \varsigma, K) = \mathrm{H}(\zeta)$
3. $(\mathbf{s_1}, \mathbf{s_2}) \in S_\eta^\ell \times S_\eta^k = \mathrm{H}(\varsigma)$
4. $\bar{\mathbf{A}} \in \mathcal{R}_q^{k \times \ell} = \mathsf{ExpandA}(\rho)$
5. $\mathbf{t} = \mathsf{INTT}(\bar{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{s_1})) + \mathbf{s_2}$
6. $(\mathbf{t_1}, \mathbf{t_0}) = \mathsf{Power2Round}_q(\mathbf{t}, d)$
7. $tr = \mathsf{CRH}(\rho || \mathbf{t_1})$
8. $pk = (\rho, \mathbf{t_1})$
9. $sk = (\rho, K, tr, \mathbf{s_1}, \mathbf{s_2}, \mathbf{t_0})$
10. **return** (pk, sk)

$\mathsf{Dilithium.Sign}(sk, M, rnd)$ ....

1. $\bar{\mathbf{A}} \in \mathcal{R}_q^{k \times \ell} = \mathsf{ExpandA}(\rho)$
2. $\mu = \mathsf{CRH}(tr || M)$
3. $\rho' = \mathsf{CRH}(K || rnd || \mu)$
4. $\kappa = 0$
5. $(\mathbf{z}, \mathbf{h}) = \perp$
6. **While** $(\mathbf{z}, \mathbf{h}) = \perp$
7. $\quad \mathbf{y} \in \tilde{S}_{\gamma_1}^\ell = \mathsf{ExpandMask}(\rho', \kappa)$
8. $\quad \mathbf{w} = \mathsf{INTT}(\bar{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{y}))$
9. $\quad \mathbf{w_1} = \mathsf{HighBits}_q(\mathbf{w}, 2\gamma_2)$
10. $\quad \tilde{c} = \mathrm{H}(\mu, \mathbf{w_1})$
11. $\quad c \in \mathcal{B}_\tau = \mathsf{SampleInBall}(\tilde{\mathsf{c}})$
12. $\quad \mathbf{z} = \mathbf{y} + c\mathbf{s_1}$
13. $\quad \mathbf{r_0} = \mathsf{LowBits}_q(\mathbf{w} - c\mathbf{s_2}, 2\gamma_2)$
14. $\quad$ **If** $(||\mathbf{z}||_\infty \geq \gamma_1 - \beta)$ or $(||\mathbf{r_0}||_\infty \geq \gamma_2 - \beta)$
15. $\quad\quad (\mathbf{z}, \mathbf{h}) = \perp$
16. $\quad$ **Else**
17. $\quad\quad \mathbf{h} = \mathsf{MakeHint}_q(-c\mathbf{t_0}, \mathbf{w} - c\mathbf{s_2} + c\mathbf{t_0}, 2\gamma_2)$
18. $\quad\quad$ **If** $(||c\mathbf{t_0}||_\infty \geq \gamma_2)$ or (# of 1's in $\mathbf{h} > \omega$)
19. $\quad\quad\quad (\mathbf{z}, \mathbf{h}) = \perp$
20. $\quad \kappa = \kappa + \ell$
21. **Return** $\sigma = (\mathbf{z}, \mathbf{h}, \tilde{c})$

$\mathsf{Dilithium.Verify}(pk, M, \sigma)$ ....

1. $\bar{\mathbf{A}} \in \mathcal{R}_q^{k \times \ell} = \mathsf{ExpandA}(\rho)$
2. $\mu = \mathsf{CRH}(\mathsf{CRH}(\rho || \mathbf{t_1}) || M)$
3. $c = \mathsf{SampleInBall}(\tilde{\mathsf{c}})$
4. $\mathbf{w}' = \mathsf{INTT}(\bar{\mathbf{A}} \circ \mathsf{NTT}(\mathbf{z}) - \mathsf{NTT}(c) \circ \mathsf{NTT}(\mathbf{t_1} \cdot 2^d))$
5. $\mathbf{w_1}' = \mathsf{UseHint}_q(\mathbf{h}, \mathbf{w}', 2\gamma_2)$
6. $\tilde{c}' = \mathrm{H}(\mu, \mathbf{w_1}')$
7. **If** $(||\mathbf{z}||_\infty < \gamma_1 - \beta)$ and (# of 1's in $\mathbf{h} < \omega$) and $(\tilde{c} = \tilde{c}')$
8. $\quad$ **Return** True
9. **Else**
10. $\quad$ **Return** False

Figure 13: Dilithium algorithms from [DKL+22].

References

[AB75] R.C. Agarwal and C.S. Burrus. Number theoretic transforms to implement fast digital convolution. *Proceedings of the IEEE*, 63(4):550–560, 1975.

[ABC+23] Melissa Azouaoui, Olivier Bronchain, Gaëtan Cassiers, Clément Hoffmann, Yulia Kuzovkova, Joost Renes, Tobias Schneider, Markus Schönauer, François-Xavier Standaert, and Christine van Vredendaal. Protecting dilithium against leakage: Revisited sensitivity analysis and improved implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):58–79, 2023.

[AHKS22] Amin Abdulrahman, Vincent Hwang, Matthias J Kannwischer, and Amber Sprenkels. Faster kyber and dilithium on the cortex-m4. In *International Conference on Applied Cryptography and Network Security*, pages 853–871. Springer, 2022.

[Ajt96] M. Ajtai. Generating hard instances of lattice problems (extended abstract). In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '96, page 99–108, New York, NY, USA, 1996. Association for Computing Machinery.

[AMI+23] Aikata Aikata, Ahmet Can Mert, Malik Imran, Samuel Pagliarini, and Sujoy Sinha Roy. Kali: A crystal for post-quantum security using kyber and dilithium. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 70(2):747–758, 2023.

[AOP+25] Amin Abdulrahman, Felix Oberhansl, Hoang Nguyen Hien Pham, Jade Philipoom, Peter Schwabe, Tobias Stelzer, and Andreas Zankl. Towards ml-kem & ml-dsa on opentitan. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 1–19. IEEE, 2025.

[BAA23] Luke Beckwith, Abubakr Abdulgadir, and Reza Azarderakhsh. A flexible shared hardware accelerator for nist-recommended algorithms crystals-kyber and crystals-dilithium with sca protection. In *Cryptographers' Track at the RSA Conference*, pages 469–490. Springer, 2023.

[Bar87a] Paul Barrett. Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 311–323. Springer, Berlin, Heidelberg, August 1987.

[Bar87b] Paul Barrett. Implementing the RSA public key encryption algorithm on a standard digital signal processor. In *Proceedings on Advances in Cryptology - CRYPTO '86*, volume 263 of *Lecture Notes in Computer Science*, pages 311–323. Springer, 1987.

[BBD+16] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong non-interference and type-directed higher-order masking. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, page 116–129, New York, NY, USA, 2016. Association for Computing Machinery.

[BC22] Olivier Bronchain and Gaëtan Cassiers. Bitslicing arithmetic/boolean masking conversions for fun and profit: with application to lattice-based kems.

IACR Transactions on Cryptographic Hardware and Embedded Systems, 2022(4):553–588, Aug. 2022.

[BDH+21] Shivam Bhasin, Jan-Pieter D'Anvers, Daniel Heinz, Thomas Pöppelmann, and Michiel Van Beirendonck. Attacking and defending masked polynomial comparison. *IACR TCHES*, 2021(3):334–359, 2021.

[BDK+21] Michiel Van Beirendonck, Jan-Pieter D'anvers, Angshuman Karmakar, Josep Balasch, and Ingrid Verbauwhede. A side-channel-resistant implementation of saber. *J. Emerg. Technol. Comput. Syst.*, 17(2), apr 2021.

[BDK+22] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber algorithm specifications and supporting documentation. Technical report, National Institute of Standards and Technology, 2022. Round 3 Submission.

[BDPA12] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The Keccak sha-3 submission. In *Proceedings of the Round 3 SHA-3 Workshop*, 2012. Retrieved: 2025-12-07.

[BGG+14] Josep Balasch, Benedikt Gierlichs, Vincent Grosso, Oscar Reparaz, and François-Xavier Standaert. On the cost of lazy engineering for masked software implementations. In *International Conference on Smart Card Research and Advanced Applications*, pages 64–81. Springer, 2014.

[BGR+21] Joppe W. Bos, Marc Gourjon, Joost Renes, Tobias Schneider, and Christine van Vredendaal. Masking Kyber: First- and higher-order implementations. *IACR TCHES*, 2021(4):173–214, 2021.

[BHH+15] Daniel J. Bernstein, Daira Hopwood, Andreas Hülsing, Tanja Lange, Ruben Niederhagen, Louiza Papachristodoulou, Michael Schneider, Peter Schwabe, and Zooko Wilcox-O'Hearn. SPHINCS: Practical stateless hash-based signatures. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 368–397. Springer, Berlin, Heidelberg, April 2015.

[BNG21] Luke Beckwith, Duc Tri Nguyen, and Kris Gaj. High-performance hardware implementation of crystals-dilithium. In *2021 International Conference on Field-Programmable Technology (ICFPT)*, pages 1–10. IEEE, 2021.

[Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 868–886. Springer, Berlin, Heidelberg, August 2012.

[CGL+24] Jean-Sébastien Coron, François Gérard, Tancrède Lepoint, Matthias Trannoy, and Rina Zeitoun. Improved high-order masked generation of masking vector and rejection sampling in dilithium. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(4):335–354, Sep. 2024.

[CGMZ23] Jean-Sébastien Coron, François Gérard, Simon Montoya, and Rina Zeitoun. High-order polynomial comparison and masking lattice-based encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 153–192, 2023.

[CGTZ23] Jean-Sébastien Coron, François Gérard, Matthias Trannoy, and Rina Zeitoun. Improved gadgets for the high-order masking of dilithium. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4), 2023.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 409–437. Springer, Cham, December 2017.

[CKRG+24] Xavier Carril, Charalampos Kardaris, Jordi Ribes-GonzáLez, Oriol Farràs, Carles Hernandez, Vatistas Kostalabros, Joel Ulises González-Jiménez, and Miquel Moreto. Hardware acceleration for high-volume operations of crystals-kyber and crystals-dilithium. *ACM Transactions on Reconfigurable Technology and Systems*, 17(3):1–26, 2024.

[CT65] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.

[D+15] Morris J Dworkin et al. Sha-3 standard: Permutation-based hash and extendable-output functions. 2015.

[DGH+23] Christoph Dobraunig, Lorenzo Grassi, Lukas Helminger, Christian Rechberger, Markus Schofnegger, and Roman Walch. Pasta: A case for hybrid homomorphic encryption. *IACR TCHES*, 2023(3):30–73, 2023.

[DH76] W. Diffie and M. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.

[DKL+22] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium algorithm specifications and supporting documentation. Technical report, National Institute of Standards and Technology, 2022. NIST PQC Selected Algorithm Submission, Version 3.02.

[DMMM23] Alessandra Dolmeta, Mattia Mirigaldi, Maurizio Martina, and Guido Masera. Implementation and integration of keccak accelerator on RISC-V for crystals-kyber. In *Proceedings of the 20th ACM International Conference on Computing Frontiers*, CF '23, page 381–382, New York, NY, USA, 2023. Association for Computing Machinery.

[DVBV22] Jan-Pieter D'Anvers, Michiel Van Beirendonck, and Ingrid Verbauwhede. Revisiting higher-order masked comparison for lattice-based cryptography: Algorithms and bit-sliced implementations. *IEEE Transactions on Computers*, 72(2):321–332, 2022.

[DZD+17] A Adam Ding, Liwei Zhang, François Durvaux, François-Xavier Standaert, and Yunsi Fei. Towards sound and optimal leakage detection procedure. In *International Conference on Smart Card Research and Advanced Applications*, pages 105–122. Springer, 2017.

[FHK+18] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, Zhenfei Zhang, et al. Falcon: Fast-fourier lattice-based compact signatures over ntru. *Submission to the NIST's post-quantum cryptography standardization process*, 36(5):1–75, 2018.

[FV12] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. IACR Cryptology ePrint Archive, Report 2012/144, 2012.

[FVR+22] Tim Fritzmann, Michiel Van Beirendonck, Debapriya Basu Roy, Patrick Karl, Thomas Schamberger, Ingrid Verbauwhede, and Georg Sigl. Masked accelerators and instruction set extensions for post-quantum cryptography. *IACR TCHES*, 2022(1):414–460, 2022.

[GGJR+11] Benjamin Jun Gilbert Goodwill, Josh Jaffe, Pankaj Rohatgi, et al. A testing methodology for side-channel resistance validation. In *NIST non-invasive attack testing workshop*, volume 7, pages 115–136, 2011.

[GLK21] Wenbo Guo, Shuguo Li, and Liang Kong. An efficient implementation of kyber. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 69(3):1562–1566, 2021.

[GS66] W. M. Gentleman and G. Sande. Fast fourier transforms: for fun and profit. In *Proceedings of the 1966 Fall Joint Computer Conference*, 1966.

[GSM17] Hannes Groß, David Schaffenrath, and Stefan Mangard. Higher-order side-channel protected implementations of keccak, 2017.

[HHLW20] Yiming Huang, Miaoqing Huang, Zhongkui Lei, and Jiaxuan Wu. A pure hardware implementation of crystals-kyber pqc algorithm through resource reuse. *IEICE Electronics Express*, 17(17):20200234–20200234, 2020.

[HKL+22] Daniel Heinz, Matthias J Kannwischer, Georg Land, Thomas Pöppelmann, Peter Schwabe, and Daan Sprenkels. First-order masked kyber on arm cortex-m4. *Cryptology ePrint Archive*, 2022.

[ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003*, pages 463–481, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[JGCS24] Arpan Jati, Naina Gupta, Anupam Chattopadhyay, and Somitra Kumar Sanadhya. A configurable crystals-kyber hardware implementation with side-channel protection. *ACM transactions on embedded computing systems*, 23(2):1–25, 2024.

[Kar63] Anatolii Karatsuba. Multiplication of multidigit numbers on automata. In *Soviet physics doklady*, volume 7, pages 595–596, 1963.

[KDVB+22] Suparna Kundu, Jan-Pieter D'Anvers, Michiel Van Beirendonck, Angshuman Karmakar, and Ingrid Verbauwhede. Higher-order masked saber. In *International Conference on Security and Cryptography for Networks*, pages 93–116. Springer, 2022.

[KNAH22] Tendayi Kamucheka, Alexander Nelson, David Andrews, and Miaoqing Huang. A masked pure-hardware implementation of kyber cryptographic algorithm. In *2022 International Conference on Field-Programmable Technology (ICFPT)*, pages 1–1. IEEE, 2022.

[Kob87] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of Computation*, 48(177):203–209, 1987.

[KRSS19] Matthias J Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking nist pqc on arm cortex-m4. 2019.

[KSFS24] Patrick Karl, Jonas Schupp, Tim Fritzmann, and Georg Sigl. Post-quantum signatures on risc-v with hardware acceleration. *ACM Transactions on Embedded Computing Systems*, 23(2):1–23, 2024.

[MBB+23] Konstantina Miteloudi, Joppe W Bos, Olivier Bronchain, Björn Fay, and Joost Renes. Pq. v. alu. e: Post-quantum risc-v custom alu extensions on dilithium and kyber. In *International Conference on Smart Card Research and Advanced Applications*, pages 190–209. Springer, 2023.

[MCL+23] Gaoyu Mao, Donglong Chen, Guangyan Li, Wangchen Dai, Abdurrashid Ibrahim Sanka, Çetin Kaya Koç, and Ray C. C. Cheung. High-performance and configurable sw/hw co-design of post-quantum signature crystals-dilithium. *ACM Trans. Reconfigurable Technol. Syst.*, 16(3), June 2023.

[MGTF19] Vincent Migliore, Benoît Gérard, Mehdi Tibouchi, and Pierre-Alain Fouque. Masking dilithium: Efficient implementation and side-channel evaluation. In *Applied Cryptography and Network Security: 17th International Conference, ACNS 2019, Bogota, Colombia, June 5–7, 2019, Proceedings 17*, pages 344–362. Springer, 2019.

[NDKV25] Quinten Norga, Jan-Pieter D'Anvers, Suparna Kundu, and Ingrid Verbauwhede. X2X: Low-randomness and high-throughput A2B and B2A conversions for d+1 shares in hardware. In *Constructive Approaches for Security Analysis and Design of Embedded Systems: First International Conference, CASCADE 2025, Saint-Etienne, France, April 2–4, 2025, Proceedings*, page 119–158, Berlin, Heidelberg, 2025. Springer-Verlag.

[NDMZ+21] Pietro Nannipieri, Stefano Di Matteo, Luca Zulberti, Francesco Albicocchi, Sergio Saponara, and Luca Fanucci. A risc-v post quantum cryptography instruction set extension for number theoretic transform to speed-up crystals algorithms. *IEEE Access*, 9:150798–150808, 2021.

[OSPG18] Tobias Oder, Tobias Schneider, Thomas Pöppelmann, and Tim Güneysu. Practical cca2-secure and masked ring-lwe implementation. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1):142–174, Feb. 2018.

[ÖY23] Sıla Özeren and Oğuz Yayla. Methods for masking crystals-kyber against side-channel attacks. In *2023 16th International Conference on Information Security and Cryptology (ISCTürkiye)*, pages 1–6. IEEE, 2023.

[PHGCGP22] Francisco Pajuelo-Holguera, José M. Granado-Criado, and Juan A. Gómez-Pulido. Fast montgomery modular multiplier using fpgas. *IEEE Embedded Systems Letters*, 14(1):19–22, 2022.

[Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the Thirty-seventh Annual ACM STOC*, pages 84–93, Baltimore, MD, USA, 2005.

[RSA78] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120–126, February 1978.

[Sch90] C. P. Schnorr. Efficient identification and signatures for smart cards. In Gilles Brassard, editor, *Advances in Cryptology — CRYPTO' 89 Proceedings*, pages 239–252, New York, NY, 1990. Springer New York.

[Sha79] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, November 1979.

[Sho94] P.W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994.

[SPOG19] Tobias Schneider, Clara Paglialonga, Tobias Oder, and Tim Güneysu. Efficiently masking binomial sampling at arbitrary orders for lattice-based crypto. In *Public-Key Cryptography – PKC 2019: 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography, Beijing, China, April 14-17, 2019, Proceedings, Part II*, page 534–564, Berlin, Heidelberg, 2019. Springer-Verlag.

[TG16] Michael Tunstall and Gilbert Goodwill. Applying tvla to public key cryptographic algorithms. *Cryptology ePrint Archive*, 2016.

[TMS24] Tolun Tosun, Amir Moradi, and Erkay Savas. Exploiting the central reduction in lattice-based cryptography. *IEEE Access*, 12:166814–166833, 2024.

[WWL+24] Nuo Wang, Liji Wu, Lei Li, Munkhbaatar Chinbat, and Xiangmin Zhang. Design and implementation of a ntt accelerator based on RISC-V for pqc kyber. In *2024 IEEE 18th International Conference on Anti-counterfeiting, Security, and Identification (ASID)*, pages 24–27, 2024.

[WZCG22] Tengfei Wang, Chi Zhang, Pei Cao, and Dawu Gu. Efficient implementation of dilithium signature scheme on fpga soc platform. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 30(9):1158–1171, 2022.

[WZZ+24] Tengfei Wang, Chi Zhang, Xiaolin Zhang, Dawu Gu, and Pei Cao. Optimized hardware-software co-design for kyber and dilithium on risc-v soc fpga. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(3):99–135, 2024.

[XL21] Yufei Xing and Shuguo Li. A compact hardware implementation of cca-secure key exchange mechanism crystals-kyber on fpga. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 328–356, 2021.

[YTO21] Yasin Yilmaz, Yavuz Selim Tozlu, and Berna Örs. Design and implementation of a 32-bit RISC-V core. page 460 – 464, 2021.

[ZSS+21] Sara Zarei, Aein Rezaei Shahmirzadi, Hadi Soleimany, Raziyeh Salarifard, and Amir Moradi. Low-latency Keccak at any arbitrary order. *IACR TCHES*, 2021(4):388–411, 2021.

[ZZW+22] Cankun Zhao, Neng Zhang, Hanning Wang, Bohan Yang, Wenping Zhu, Zhengdong Li, Min Zhu, Shouyi Yin, Shaojun Wei, and Leibo Liu. A compact and high-performance hardware architecture for crystals-dilithium. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 270–295, 2022.